数字交流电压表的设计 实验报告

成绩_____

1 设计要求
2 设计方案分析
3 仿真电路设计
3.1 单片机最小系统
3.2 稳压电路及信号发生电路
3.3 按键电路
3.4 信号处理电路
3.5 LCD 显示电路
3.6 信号发生器电路
3.7 无源蜂鸣器电路
4 仿真电路调试
4.1 整体系统
4.2 效果展示
5 软件程序设计
5.1 概述及主程序
1. 调用及初始化阶段
2. 主循环逻辑（while(1)）按以下顺序循环执行：
3. 定时器 0 中断服务程序（void timer0_isr(void) interrupt 1）
4. 定时器 1 中断服务程序（void timer1_isr(void) interrupt 3）
5.2 量程切换程序
5.3 ADC 数据处理程序
5.4 模拟量通道选择程序
5.5 ADC 读取程序
6 硬件电路调试
6.1 电路连接
6.2 结果测试
7 设计心得
8 附录
8.1 完整仿真电路图
8.2 完整 PCB 电路图
8.3 完整 PCB-Layoyt
8.4 完整仿真程序代码

1 设计要求

以 MCS-51 单片机为核心，设计一个交流数字电压表，交流信号由变压器提供，二次侧 0-5VAC，匝数比 200:1。测量值显示在 LCD 显示器上，被测电压由 8 位 AD 转换器将模拟量转换成数字量信号有：启动转换信号（START，输出），转换结束（EOC，输入）和读控制信号（OE，低电平有效）；另外，可以选择测量量程，例如通过单片机输出一个 3 位二进制数来选择不同的前置放大器增益。

2 设计方案分析

本设计主要包括六个模块：电源稳压模块，信号处理模块、LCD 显示模块、信号发生器模块、按键交互模块、时钟产生模块。

自行设计的 PCB 如图：

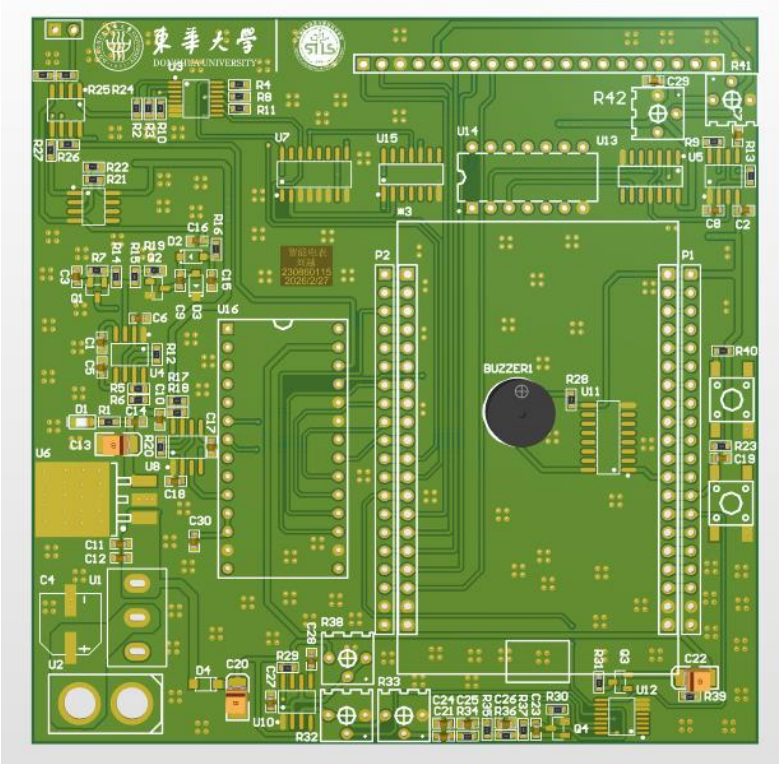


图 1 自制 STC89C52RC 扩展板

主要设计方案描述如下：

硬件层：

8 位分辨率 8 通道 ADC 芯片 ADC0808：读取外部输入的模拟信号以及自行产生的四种信号。

LCD12864：显示当前选择模式以及对应模式的输出（模式 1：示波器，模式 2：交流电压表，模式 3：信号发生器，模式 4：音乐播放器）

运放信号处理电路：把外部输入的-5V 到+5V 的交流信号经过电压跟随器，运算乘法器，运算加法器整合为 0 到 5V 的新的交流信号输入到 ADC 芯片中，满足 ADC0808 的输入电压需求。并且在当中加入模拟开关电路用于软件选择电压表的量程（有三种，1X，2X，4X）

信号发生器电路：利用 555 定时器产生交流信号，通过模拟开关选择信号的类型（方波，锯齿波，三角波，正弦波）

按键交互电路：共有 KEY1 和 KEY2 两个按键，KEY1 用于切换运行模式，采用定时器消抖，KEY2 用于切换交流电压表的量程，为了节省 51 定时器的 IO 口资源，采用双 D 触发器组成的硬件消抖电路。

软件逻辑：

系统主程序采用 **循环结构** 作为整体框架。

程序执行流程为：

1. 系统初始化
2. 按键检测
3. 判断当前运行模式
4. 执行对应功能模块
5. 更新显示数据

通过这种结构可以保证系统实时响应用户操作。

3 仿真电路设计

3.1 单片机最小系统

由于我们采用最小系统板+自制扩展板的硬件结构，所以对最小系统不再进行仿真，最小系统如图所示，包括稳压模块，外部晶振电路，复位按键电路，引脚接口等

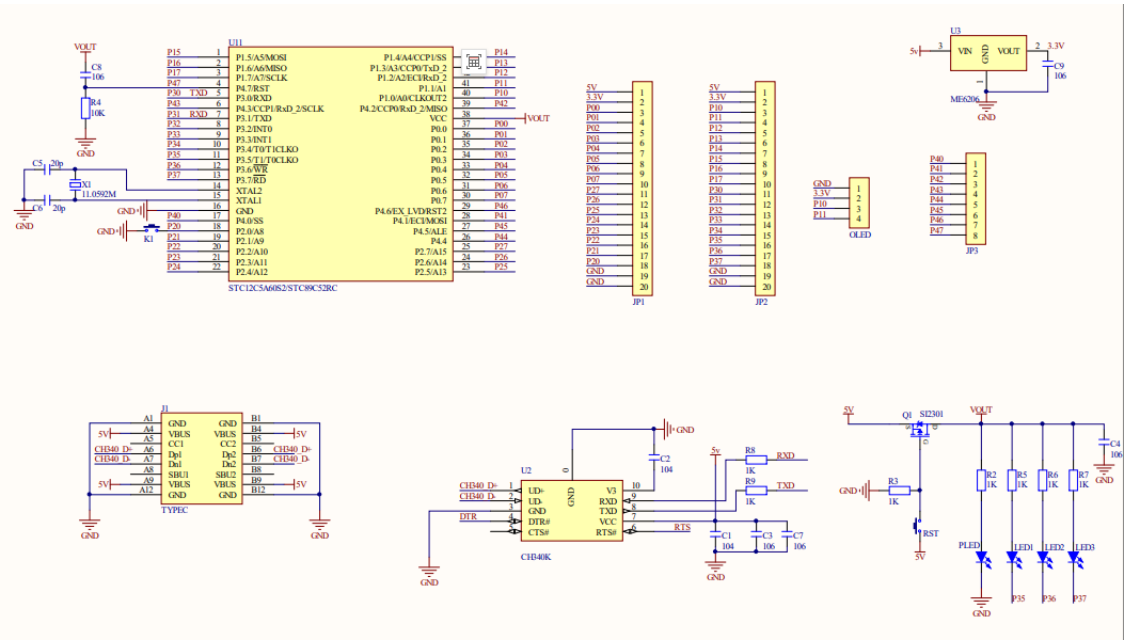


图 2 单片机最小系统电路图

3.2 稳压电路及信号发生电路

由于 Proteus 内置的电源相关芯片过少，且本人对于各类常见电源芯片已有一定了解，无需仿真也可保证打板成功，所以稳压部分不再进行仿真。

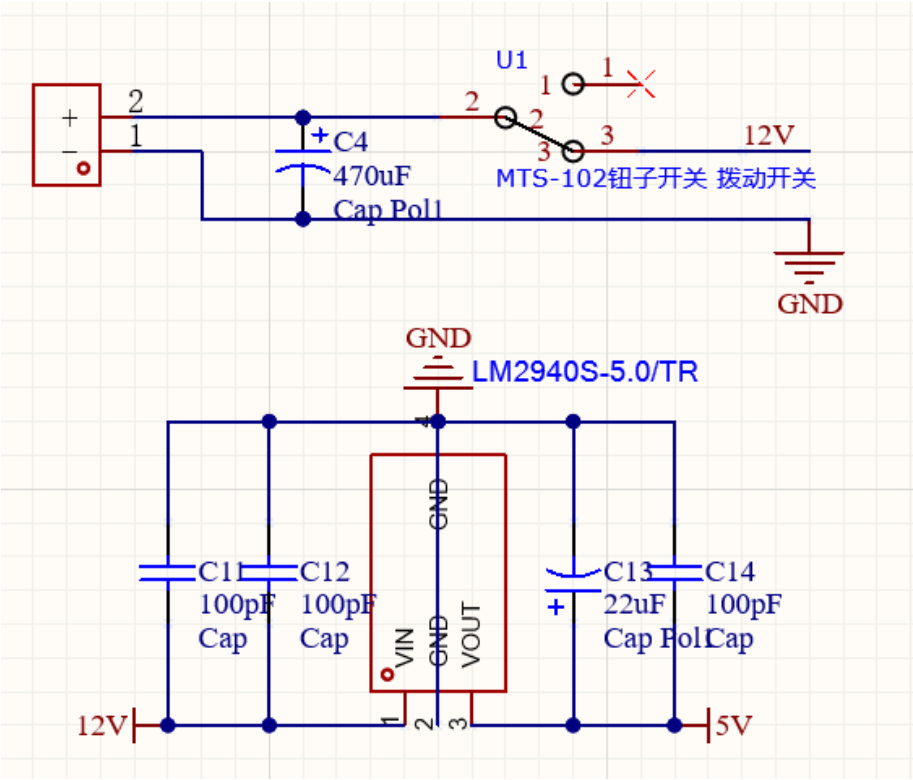


图 3 12V 转 5V 稳压电路

由于后续电路设计中需要用到三个时钟信号（500Hz,64KHz,640KHz）,为了节省 8051 宝贵的定时器资源，我们选择使用 555 定时器而不是定时器产生时钟信号：

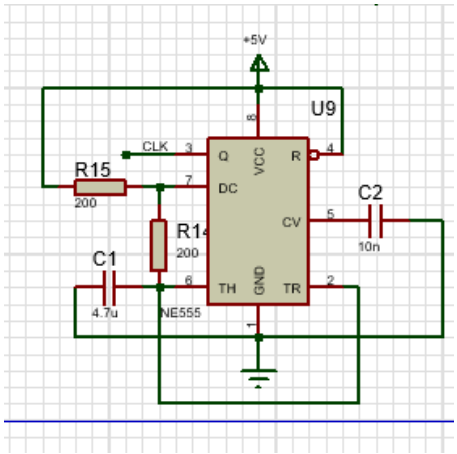


图 4 时钟信号产生电路

3.3 按键电路

本项目中一共用到两种按键，KEY1 用于切换电路模式，KEY2 用于切换电压表量程。

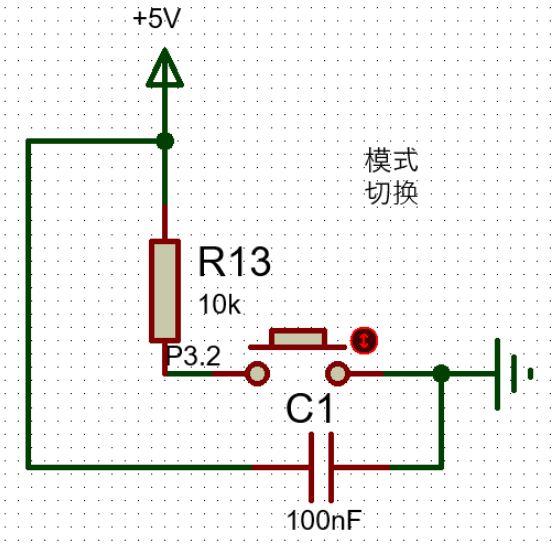


图 5 模式切换按键

对于 KEY1，外接一个上拉电阻使其默认状态下保持高电位，当按键按下后，引脚与 GND 短路，被置位低电平。用电平的高低变化来体现按键的状态。在按键两端并联一个 MLCC 进行了一个简单的硬件消抖，在代码中还进行了定时器消抖处理。

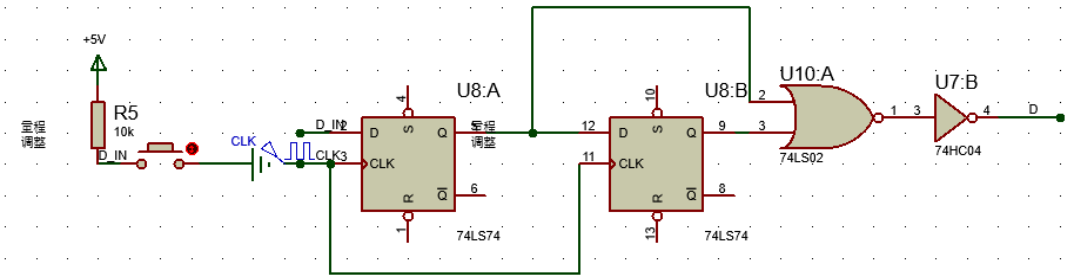


图 6 量程切换按键消抖电路

对于 KEY2，为了节省 IO 口，采用硬件电路进行量程的切换。首先通过双 D 触发器进行硬件消抖。

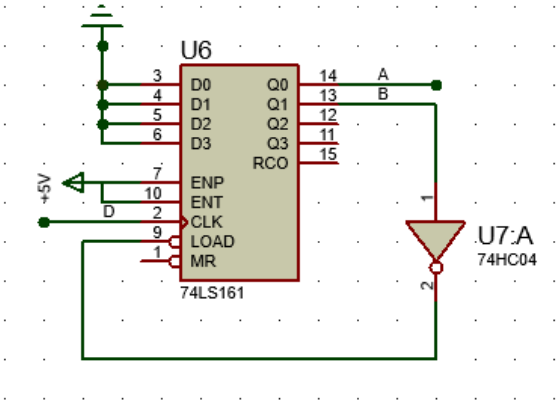


图 7 量程切换状态码生成电路

消抖后的按键信号作为 74161 计数器的时钟信号送入，通过同步置位法让 Q0 和 Q1 在 00，01，10 三个数之间循环，实现模式状态码的生成。Q0 和 Q1 一方面与 P2.6，P2.7 相连，使得 51 单片机可以实时监控量程的状态，另一方面与模拟开关相连，完成信号放大倍数的切换。

3.4 信号处理电路

由于 ADC0808 芯片对于输入的模拟信号具有较窄的输入范围（0 到 5V），为了扩展输入信号范围到-5V 到 5V，我们对输入信号进行了一系列的调整处理。

首先我们采用一个电压跟随器将信号输入端与后级电路进行缓冲隔离：

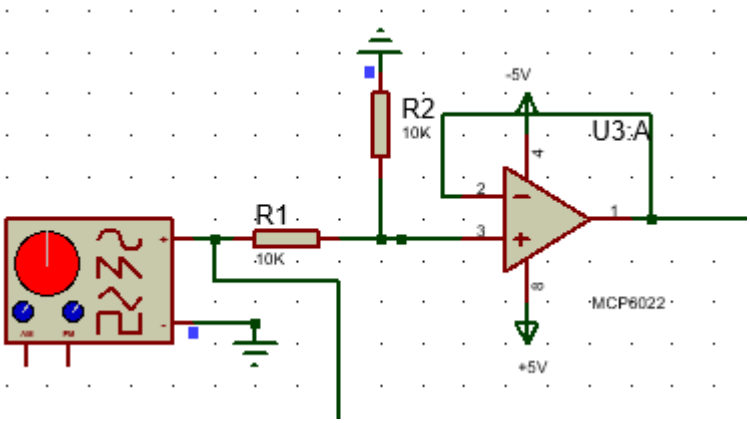


图 8 电压跟随器

然后我们采用一个运算加法器将输入信号除以 2 后再加上 2.5V 的基准直流信号，由于题目要求可以通过软件调节电压表的量程，我们在这里加入一个模拟开关用于选择输入端的电阻阻值，根据公式可得 R6 与输入端电阻的商即为信号的放大倍数，取输入端电阻分别为 10K，5K，2.5K 即可得到 1X,2X,4X 三种量程。

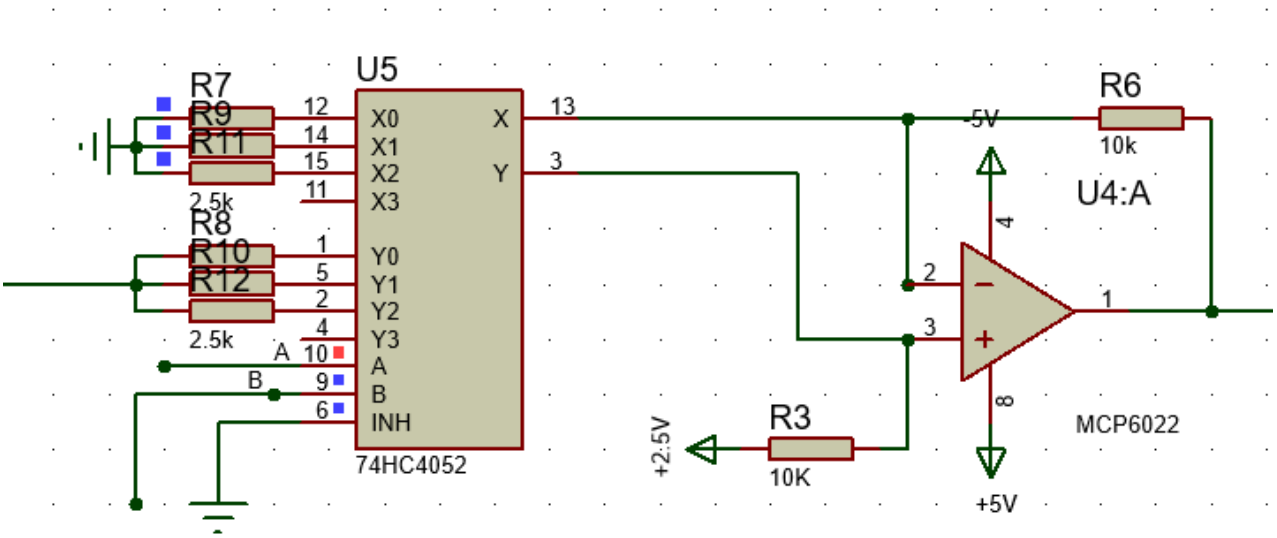


图 9 可调放大倍数的运算加法器

将地址位 A、B 分别接到 P2.6，P2.7 引脚。量程与引脚电平的关系如下表所示：

表 1 量程状态码

	P2.7=0	P2.7=1
P2.6=0	1X	2X
P2.6=1	4X	

3.5 LCD 显示电路

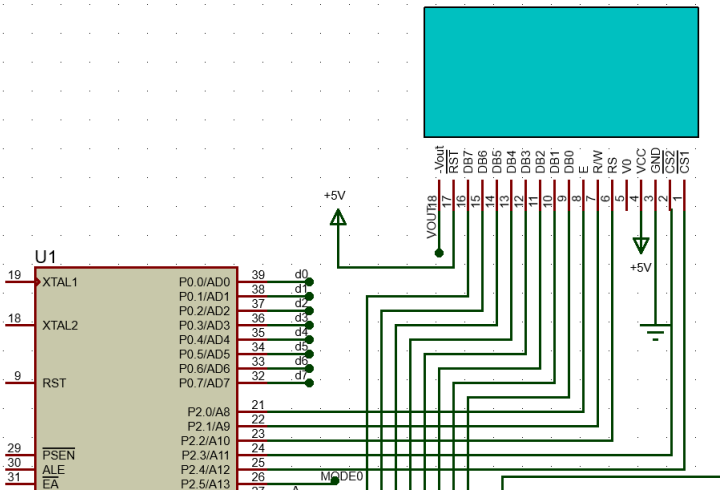


图 10 仿真时的 LCD12864 电路

在仿真时不清楚 V0 与 VOUT 这两个引脚的作用，悬空时屏幕仍然可以正常显示就没管。打板后才得知这两个引脚是用来调节屏幕的对比度的。VOUT 引脚同时也作为了运算放大器负压的来源。

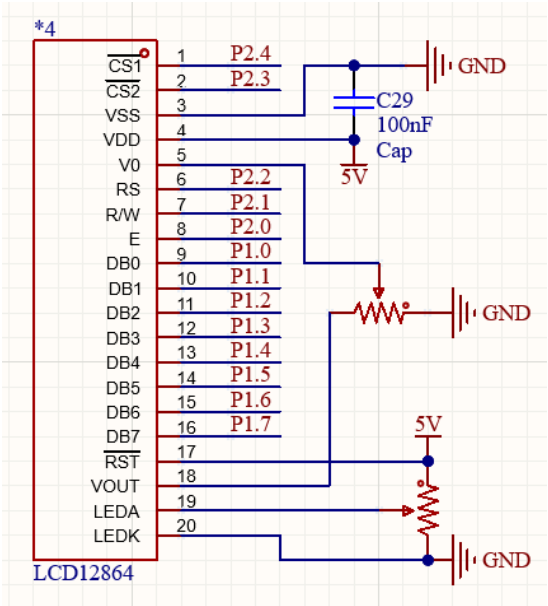


图 11 实际的 LCD12864 电路

重新打板后，在屏幕电路中添加了两个电位器用于调节屏幕的背光亮度与屏幕的对比度。

3.6 信号发生器电路

利用 555 定时器构成可产生四种波形的可调频率与幅值的信号发生器电路。通过模拟开关选择所需要发生波形的种类。

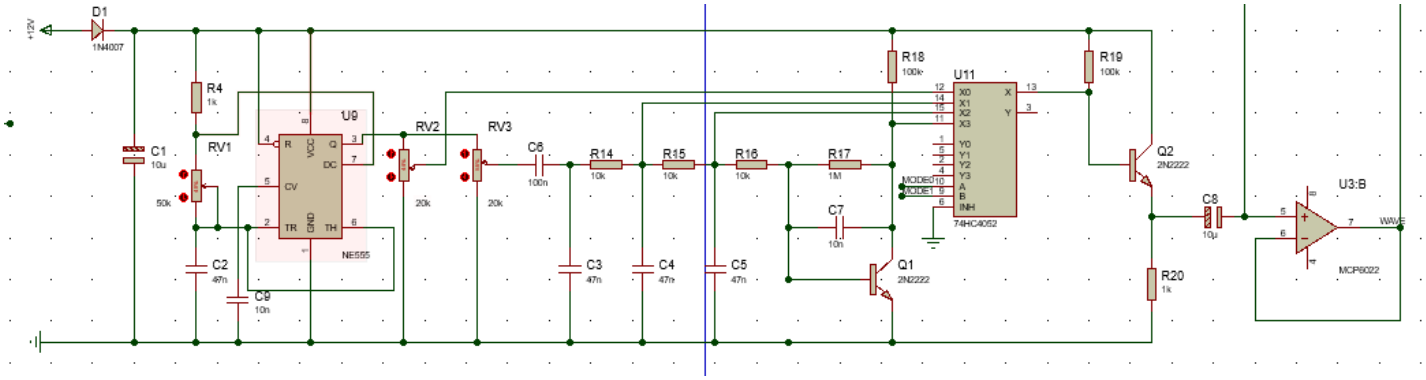


图 12 可产生四种波形的可调频率与幅值的信号发生器电路

3.7 无源蜂鸣器电路

由于单片机的 IO 口驱动能力弱，所以加入了一个达林顿管作为缓冲器，加强了 IO 口的驱动能力。

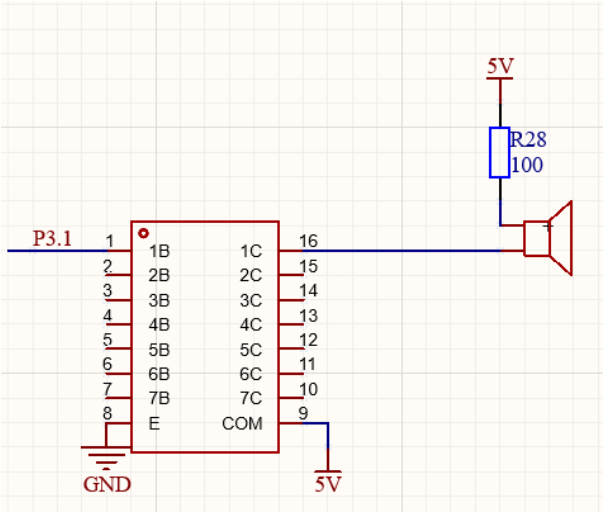


图 13 达林顿管蜂鸣器缓冲电路

4 仿真电路调试

4.1 整体系统

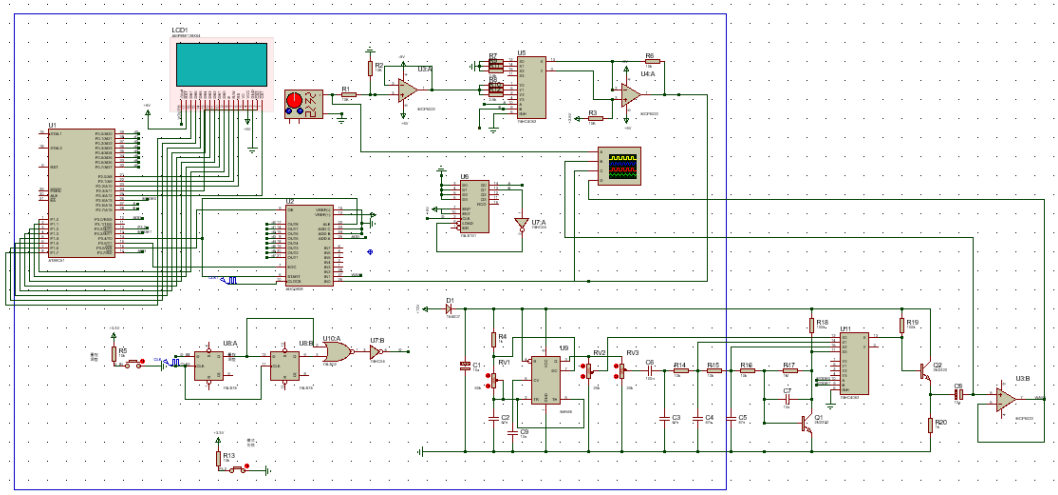


图 14 系统仿真电路图

4.2 效果展示

使用例程调试该部分电路，LCD 显示结果与预期一致，该部分仿真电路设计可以实现功能，调试结果见下图。

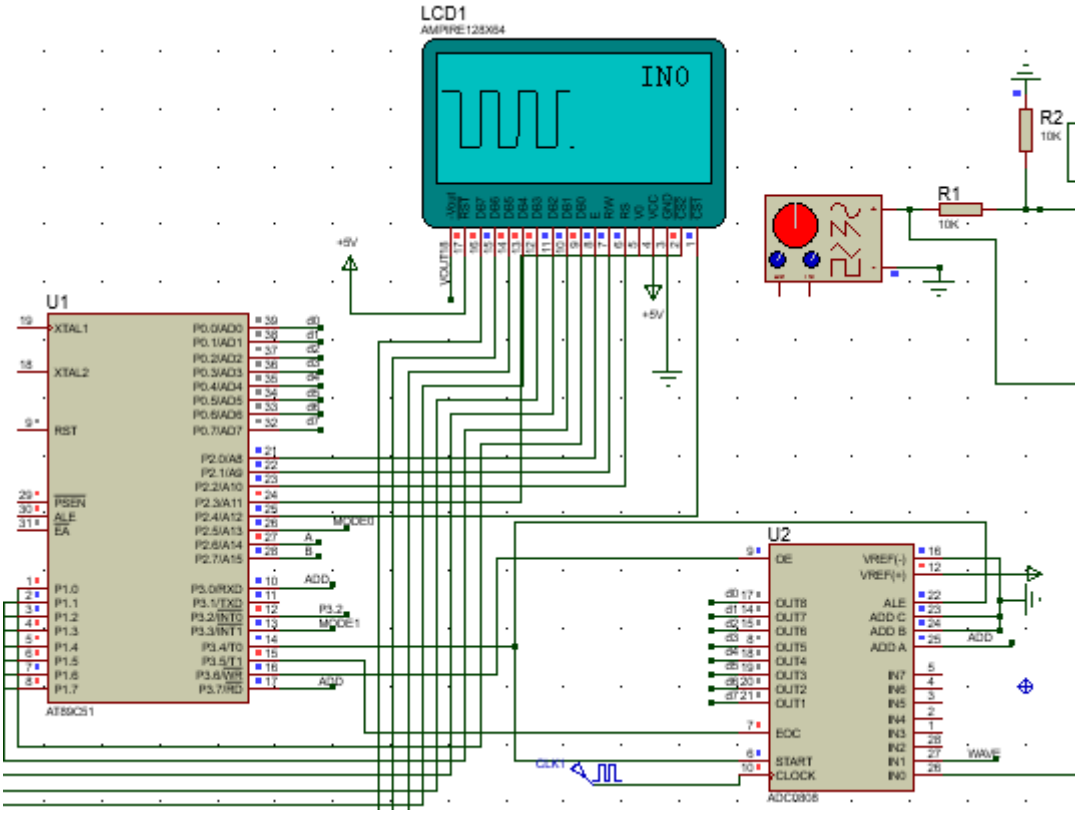


图 15 示波器模式

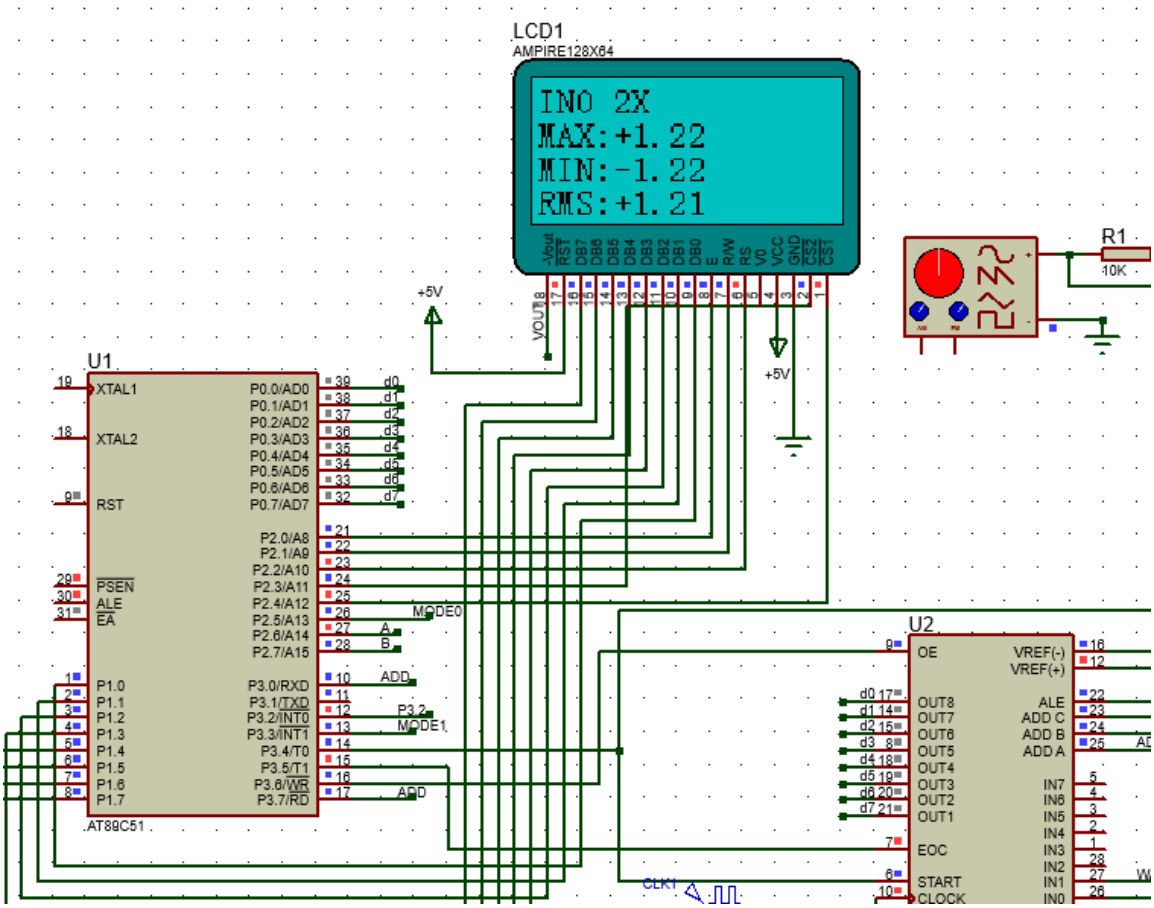


图 16 2X 量程下的交流电压表模式

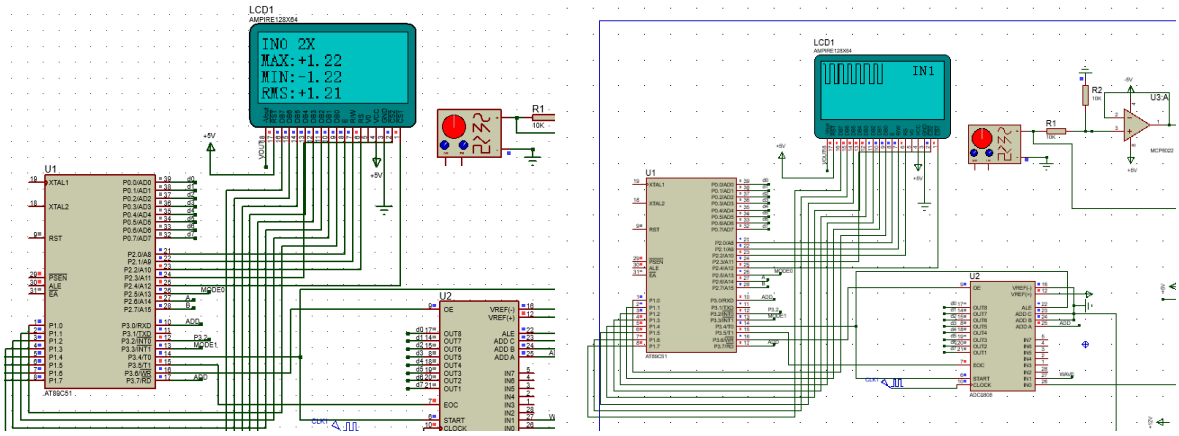


图 17 信号发生器的方波输出模式

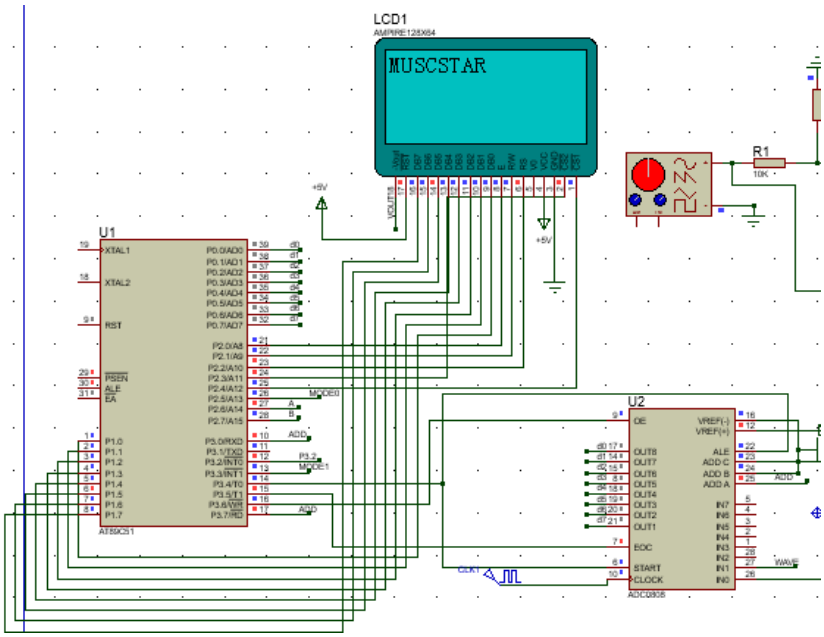


图 18 音乐播放器模式

5 软件程序设计

5.1 概述及主程序

1. 调用及初始化阶段

完成对 font 库，LCD 库，reg51 库的调用；完成对基本变量与函数的定义与声明；完成对常量的宏定义。

2. 主循环逻辑（while(1)）按以下顺序循环执行：

1.1 按键检测（区分长按与短按）：长按切换外部输入信号模式/内部信号发生器模式，短按切换示波器模式/交流电压表模式/音乐播放器模式

```c
if (KEY == 0) {
    key_count++;
    if (!long_press_handled && key_count >= KEY_THRESHOLD) {
        channel ^= 1;
        CH_SEL = !CH_SEL;
        select_adc_channel(channel);

        if (channel == 1) {
            wave_index = 0;
            current_range_code = ((uint8_t)RANGE1 << 1) | (uint8_t)RANGE0;
            prev_range_code = current_range_code;
            set_waveform_pins(wave_index);
        }
        LCD12864_Clear();
        long_press_handled = 1;
    }
} else {
    if (key_count > 0 && key_count < KEY_THRESHOLD) {
        if (mode == 1) mode = 2;
        else if (mode == 2) mode = 3;
        else mode = 1;

        LCD12864_Clear();
    }
    key_count = 0;
    long_press_handled = 0;
}
```

图 19 按键检测

1.2 模式切换：编写各模式的具体代码

```c
if (mode == 1) {
    /* Oscilloscope mode */
    old_Data = 0;
    old_page = 0;

    for (i = 0; i < COLLECT_NUM; i++) {
        sample_value = GatAdcValue();
        col = i;
        row = sample_value / 4;
        page = row / 8;
        Data = (uint8_t)(0x01 << (row - (page * 8)));

        refresh_one_list(col);

        if (i != 0) {
            temp_Data2 = Data;
            temp_page = page;

            if (page == old_page) {
                if (Data > old_Data) {
                    t = Data >> 1;
                    while (t != old_Data) {
                        t = t >> 1;
                        Data = (uint8_t)(Data | (Data >> 1));
                    }
                } else if (Data < old_Data) {
                    t = Data << 1;
                    while (t != old_Data) {
                        t = t << 1;
                        Data = (uint8_t)(Data | (Data << 1));
                    }
                }
            } else if (page > old_page) {
                t = Data;
```

图 20 示波器模式

3. 定时器 0 中断服务程序（void timer0_isr(void) interrupt 1）

作为音乐播放器模式的蜂鸣器时钟源。

```
void timer0_isr(void) interrupt 1
{
    unsigned int r = t0_reload;
    TH0 = (unsigned char)(r >> 8);
    TL0 = (unsigned char)(r & 0xFF);
    BUZZER = !BUZZER;
}
```

图 21 定时器 0

4. 定时器 1 中断服务程序（void timer1_isr(void) interrupt 3）

配合 static void systick_init(void)函数作为系统时钟。

```
void timer1_isr(void) interrupt 3
{
    TH1 = (unsigned char)(T1_RELOAD >> 8);  /* 0xFC */
    TL1 = (unsigned char)(T1_RELOAD & 0xFF);/* 0x66 */
    g_ms++;
}

static void systick_init(void)
{
    /* Timer1 Mode1 16-bit */
    TMOD = (TMOD & 0x0F) | 0x10;

    TH1 = (unsigned char)(T1_RELOAD >> 8);
    TL1 = (unsigned char)(T1_RELOAD & 0xFF);

    ET1 = 1;  /* enable Timer1 interrupt */
    EA  = 1;  /* global enable */
    TR1 = 1;  /* start Timer1 */
}
```

图 22 系统时钟

5.2 量程切换程序

读取外部电路产生的量程状态码确定量程，对电压显示做出修正。

```
static uint8_t get_range_factor(void)
{
    uint8_t bits = (P2 >> 6) & 0x03;
    switch (bits) {
    case 0x00: return 1; /* 00: 1X */
    case 0x02: return 4; /* 10: 4X */
    case 0x01: return 2; /* 01: 2X */
    default:   return 1;
    }
}
```

图 23 量程切换

5.3 ADC 数据处理程序

将读取到的 8 位数字量重新转化为电压模拟量。

```
static int adc_to_mV(uint8_t adc_val, uint8_t ch)
{
    uint32_t v_adc_mV;
    long temp;
    v_adc_mV = (uint32_t)adc_val * 5000UL / 255UL;

    if (ch == 0) {
        uint8_t range = get_range_factor();
        temp = ((long)v_adc_mV - 2500L) * 2L;
        temp = temp / (long)range;
        return (int)temp;
    } else {
        return (int)v_adc_mV;
    }
}
```

图 24 ADC 数据处理

5.4 模拟量通道选择程序

根据按键扫描结果（判定为长按还是短按），调用后进行模拟量通道的切换

```
static void set_waveform_pins(uint8_t wf)
{
    switch (wf & 0x03) {
    case 0: WAVE0 = 0; WAVE1 = 0; break;
    case 1: WAVE0 = 1; WAVE1 = 0; break;
    case 2: WAVE0 = 0; WAVE1 = 1; break;
    default: WAVE0 = 1; WAVE1 = 1; break;
    }
}
```

图 25 模拟量通道选择

5.5 ADC 读取程序

根据 ADC0808 的时序编写读取程序

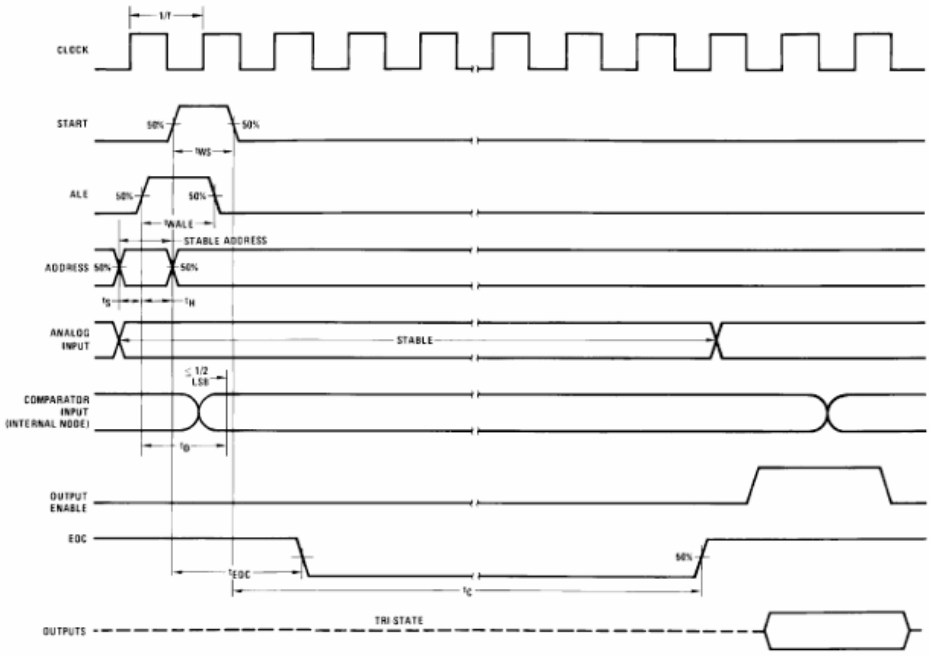


图 26　ADC0808 时序

```
uint8_t GatAdcValue()
{
    uint8_t temp;
    START = 1;
    START = 0;
    OE = 1;
    while (EOC == 0);
    temp = ADC_VALUE;
    OE = 0;
    return temp;
}
```

图 27　ADC 读取时序

6　硬件电路调试

6.1 电路连接

总实物电路图连接如下图所示。

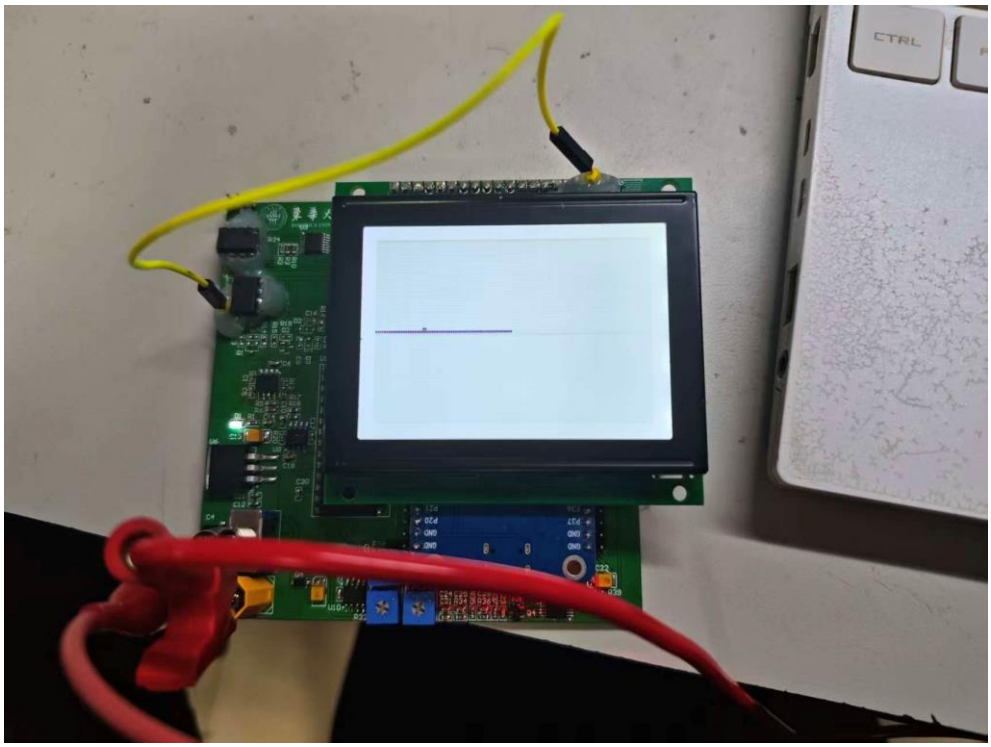


图 28 实物连接图

所用元器件清单如下表。

表 2 实物电路元器件清单

元器件	数量	单位
STC89C52RC 最小系统板	1	块
STC89C52RC 自制扩展空板	1	块
各阻值电阻	若干	个
各容值电容	若干	个
MTS102 开关	1	个
LM2940 稳压芯片	1	个
LCD12864	1	块
LF353	2	个
NE555	4	个
XT30	1	个
排针、排母	若干	个
LED0805	1	个
20 千欧电位器	3	个
50 千欧电位器	2	个
按键	2	个
5V 无源蜂鸣器	1	个
ULN2003A	1	个
74LS161	1	个
74LS02	1	个
74HC04	1	个
74HC4052	2	个
S8050	3	个
S8055	1	个

ADC0808	1	个
1N4007	1	个
1N4148W	2	个

6.2 结果测试

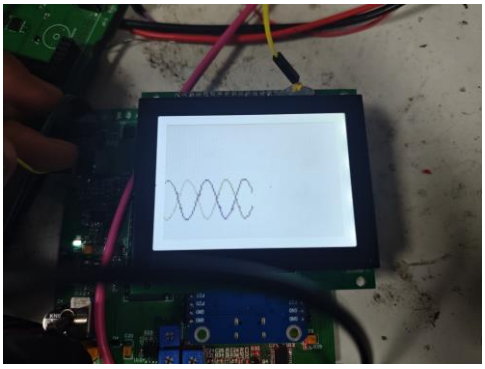


图 29 示波器模式

图 30 电压表模式（三种量程）

图 31 信号发生器模式（方波）

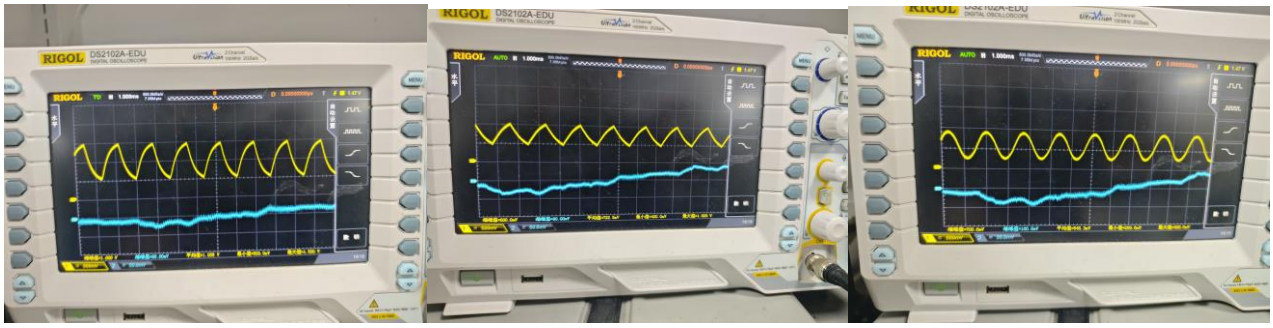


图 32 信号发生器模式（锯齿波、三角波、正弦波）

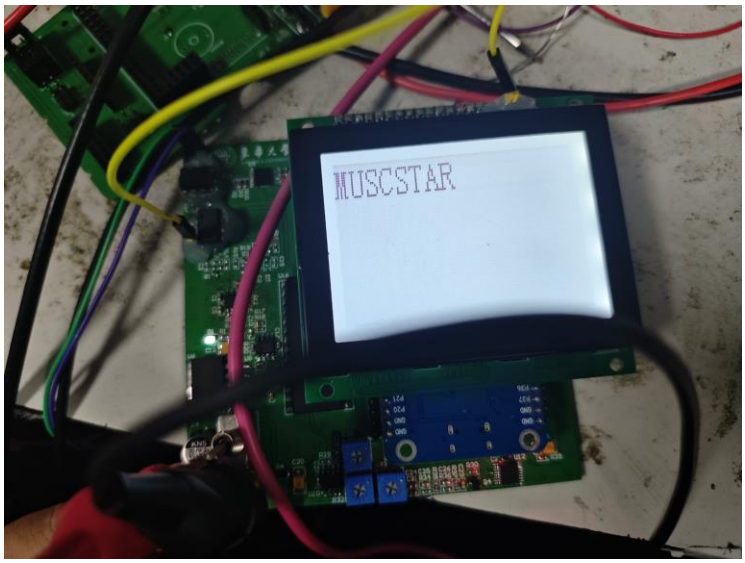


图 22 信号发生器模式（方波）

7 设计心得

通过本次课程设计，我以 MCS-51 单片机为核心，完成了一个集示波器、交流电压表、信号发生器和音乐播放器等多种功能于一体的综合系统设计。在整个设计过程中，需要同时完成硬件电路设计、软件程序编写、系统调试以及 PCB 制作等工作，使我对单片机系统开发的完整流程有了更加深入的理解。

在硬件设计方面，本系统主要包括信号处理电路、ADC 采样电路、LCD 显示电路、按键控制电路以及信号发生器电路等模块。由于 ADC0808 只能接受 0~5V 的输入电压范围，因此需要通过运算放大器电路对输入信号进行处理，将原本的±5V 交流信号转换为 0~5V 范围内的信号再送入 ADC 进行采样。这一过程让我更加深入地理解了运算放大器在信号调理中的应用。同时，在量程切换设计中，通过模拟开关与计数器电路实现了 1X、2X、4X 三种放大倍数的切换，使系统具有更好的测量范围和灵活性。

在软件设计方面，本系统采用模块化编程思想，将程序划分为 ADC 采集、数据处理、LCD 显示、按键检测和功能模式控制等多个模块。主程序通过循环结构不断检测按键状态并切换不同的工作模式，实现示波器模式、电压表模式和音乐播放模式等功能。在电压表模式中，通过对采集到的 ADC 数据进行运算处理，计算出电压的最大值、最小值以及有效值（RMS），并实时显示在 LCD12864 上。这一过程让我更加熟悉了数据处理算法在嵌入式系统中的应用。

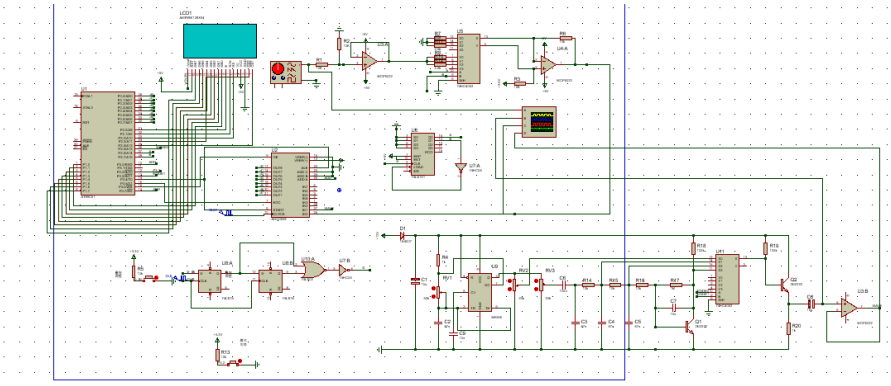
在系统调试阶段，也遇到了一些问题。例如在早期测试中，ADC 采样结果与实际电压存在一定误差，通过检查发现是信号偏置和比例计算存在问题。通过重新分析电路原理并修改软件计算公式后，最终实现了正确的电压测量结果。此外，在 LCD 显示电路中，最初没有正确连接对比度调节电位器，导致显示效果不理想，经过查阅资料和重新设计电路后才解决了这一问题。这些问题的解决过程让我深刻体会到理论知识与实际工程之间的差异，也提高了我的问题分析与调试能力。

通过本次课程设计，我不仅加深了对单片机系统结构、ADC 数据采集、信号处理以及 LCD 模块使用的理解，还提高了硬件设计与软件调试的综合能力。同时也让我认识到，在工程项目中合理规划时间、认真进行系统调试以及保持耐心的重要性。虽然在实现过程中遇到了不少困难，但通过不断尝试和修改，最终成功完成系统功能，实现了预期目标，这让我获得了很大的成就感。

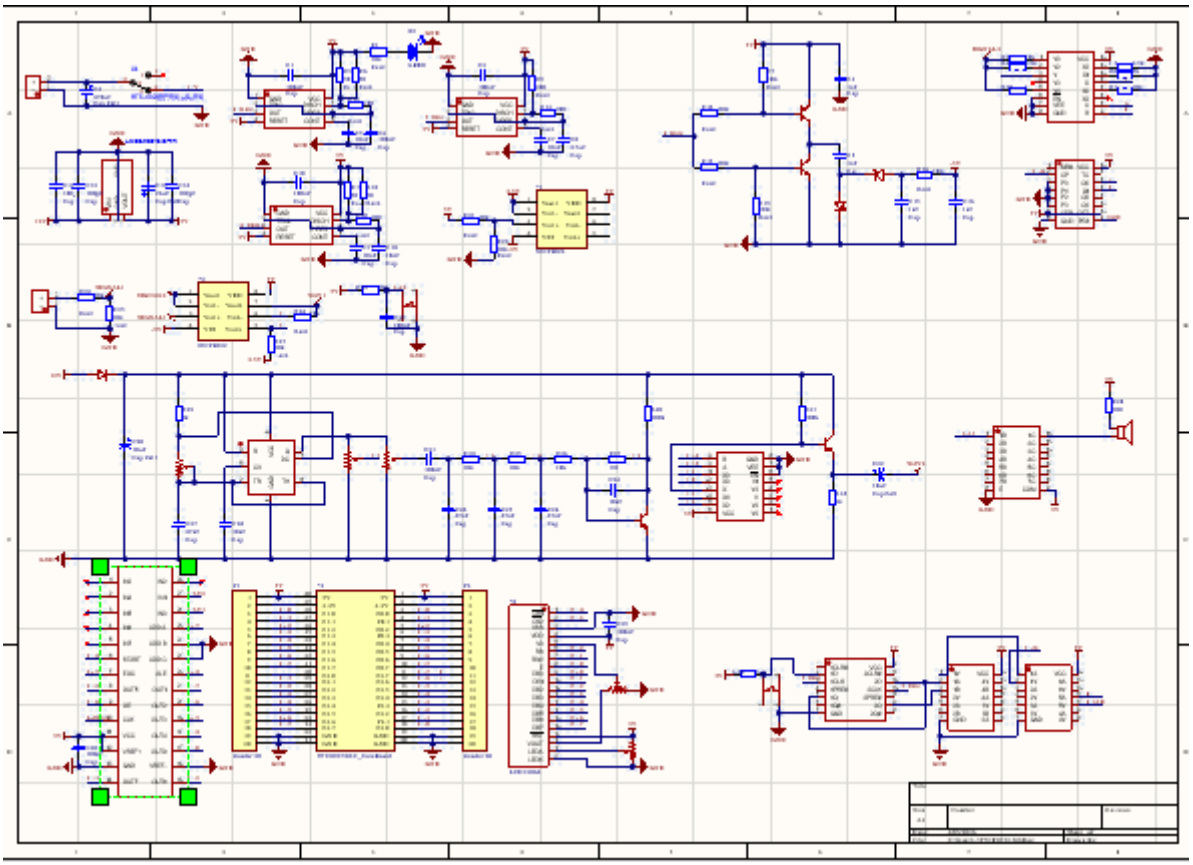
总体来说，本次课程设计将课堂上学习到的理论知识与实际工程实践相结合，使我对嵌入式系统设计有了更加全面的认识，也为今后进一步学习单片机与嵌入式系统开发打下了良好的基础。

8 附录

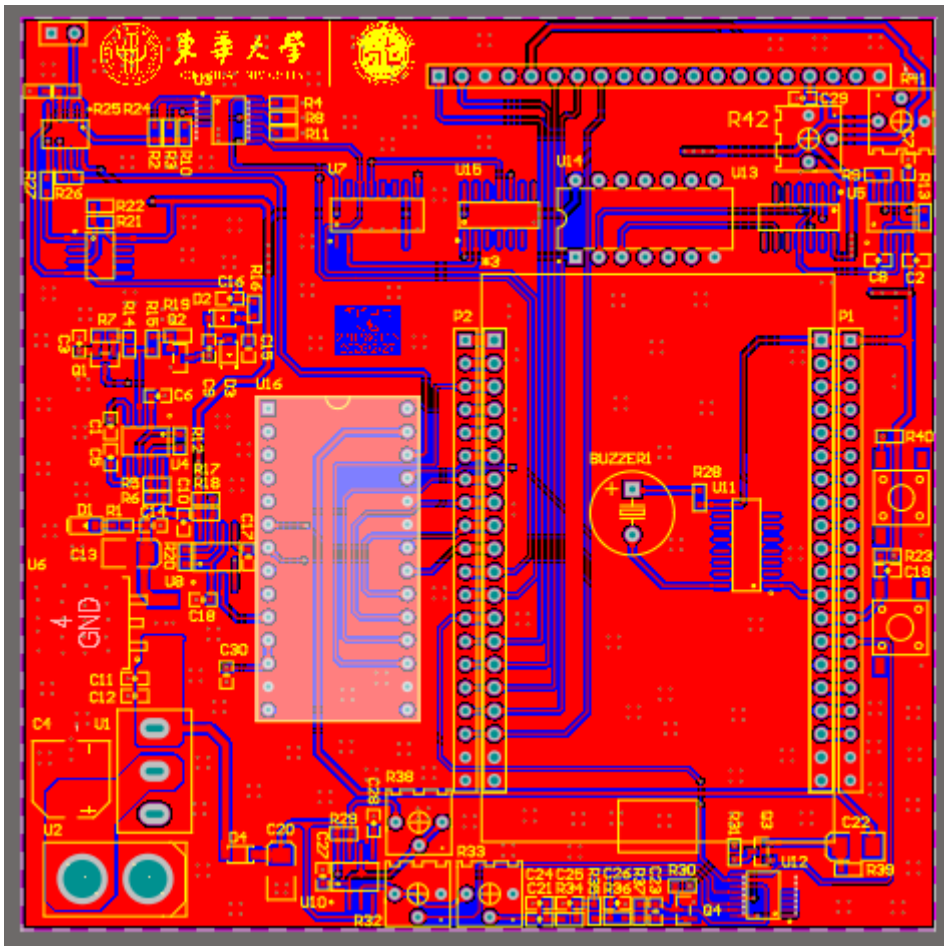
8.1 完整仿真电路图



8.2 完整 PCB 电路图



8.3 完整 PCB-Layoyt



8.4 完整仿真程序代码

```
-----------------------------------Main.c-----------------------------------
#include "reg51.h"
#include "LCD12864.h"
#include "font.h"

/* font.c defines the asc2_1608 glyph array */
#include "font.c"

/* ========================================================================== */
/* ADC0808 and control lines                                                  */
/* ========================================================================== */

#define ADC_VALUE P0

sbit START = P3^4;
sbit OE    = P3^6;
sbit EOC   = P3^5;
/* Long-press P3.2 then toggle P3.7 to switch between IN0 and IN1 */
sbit CH_SEL = P3^7;

#define COLLECT_NUM 64
```

```
/* Key and mode control */
sbit KEY = P3^2;
static uint8_t mode = 1;    /* 1=oscilloscope, 2=voltmeter, 3=music */
static uint8_t channel = 0;  /* 0=IN0, 1=IN1 */
static uint8_t key_count = 0;
static uint8_t long_press_handled = 0;

/* IN1 waveform selection */
static uint8_t wave_index = 0;
static uint8_t prev_range_code = 0;

sbit WAVE0 = P2^5;
sbit WAVE1 = P3^3;

sbit RANGE0 = P2^6;
sbit RANGE1 = P2^7;

/* Buzzer */
sbit BUZZER = P3^1;

/* Long-press threshold (loop based,保持你原逻辑) */
#define KEY_THRESHOLD 5

/* ==================================================================== */
/* Range and ADC conversion                                            */
/* ==================================================================== */

static uint8_t get_range_factor(void)
{
   uint8_t bits = (P2 >> 6) & 0x03;
   switch (bits) {
   case 0x00: return 1; /* 00: 1X */
   case 0x02: return 4; /* 10: 4X */
   case 0x01: return 2; /* 01: 2X */
   default:   return 1;
   }
}

void select_adc_channel(uint8_t ch)
{
   P1 = (P1 & 0xF8) | (ch & 0x07);
}

static uint16_t isqrt32ul(unsigned long x)
{
   unsigned long op = x;
   unsigned long res = 0;
```

```
    unsigned long one = 1UL << 30;
    while (one > op) one >>= 2;
    while (one != 0) {
        if (op >= res + one) {
            op -= res + one;
            res = res + 2 * one;
        }
        res >>= 1;
        one >>= 2;
    }
    return (uint16_t)res;
}

static int adc_to_mV(uint8_t adc_val, uint8_t ch)
{
    uint32_t v_adc_mV;
    long temp;
    v_adc_mV = (uint32_t)adc_val * 5000UL / 255UL;

    if (ch == 0) {
        uint8_t range = get_range_factor();
        temp = ((long)v_adc_mV - 2500L) * 2L;
        temp = temp / (long)range;
        return (int)temp;
    } else {
        return (int)v_adc_mV;
    }
}

static void set_waveform_pins(uint8_t wf)
{
    switch (wf & 0x03) {
    case 0: WAVE0 = 0; WAVE1 = 0; break;
    case 1: WAVE0 = 1; WAVE1 = 0; break;
    case 2: WAVE0 = 0; WAVE1 = 1; break;
    default: WAVE0 = 1; WAVE1 = 1; break;
    }
}

static void voltage_to_str(int v_mV, char *buf)
{
    char sign = '+';
    long val = v_mV;
    int integer, fractional, d1, d2;

    if (val < 0) { sign = '-'; val = -val; }
    integer = (int)(val / 1000);
```

```
    fractional = (int)(val % 1000);
    d1 = fractional / 100;
    d2 = (fractional % 100) / 10;

    buf[0] = sign;
    buf[1] = (char)((integer % 10) + '0');
    buf[2] = '.';
    buf[3] = (char)(d1 + '0');
    buf[4] = (char)(d2 + '0');
    buf[5] = '\0';
}

/* ==================================================================== */
/* LCD text helpers                                                     */
/* ==================================================================== */

static void LCD_ShowChar16x8(uint8_t page, uint8_t col, uint8_t ch)
{
    uint8_t j, i, list;
    list = col;
    if (ch < ' ' || ch > '~') return;
    ch = ch - ' ';
    for (j = 0; j < 8; j++) {
        for (i = 0; i < 2; i++) {
            write_one_list((uint8_t)(page + i), (uint8_t)(list + j),
                           asc2_1608[(uint8_t)ch][j * 2 + i]);
        }
    }
}

static void LCD_ShowStr16x8(uint8_t line, uint8_t col, uint8_t *str)
{
    uint8_t page, list;
    list = col;
    switch (line) {
    case 1: page = 0; break;
    case 2: page = 2; break;
    case 3: page = 4; break;
    case 4: page = 6; break;
    default: return;
    }
    while (*str) {
        if (*str < ' ' || *str > '~') { str++; continue; }
        if (list < 128) LCD_ShowChar16x8(page, list, *str);
        list += 8;
        if (list > 127) {
            page += 2;
```

```
            list = 0;
            if (page > 6) return;
        }
        str++;
    }
}

/* ========================================================================== */
/* ADC sample                                                                 */
/* ========================================================================== */

uint8_t GatAdcValue()
{
    uint8_t temp;
    START = 1;
    START = 0;
    OE = 1;
    while (EOC == 0);
    temp = ADC_VALUE;
    OE = 0;
    return temp;
}

/* ========================================================================== */
/* System tick timer (Timer1) - 1ms tick                                      */
/* ========================================================================== */

#define FOSC_HZ 11059200UL
#define TIMER_TICK_HZ (FOSC_HZ/12UL) /* 921600 Hz */

/* 1ms 需要计数约 921.6 -> 922 */
#define T1_COUNTS_1MS 922U
#define T1_RELOAD (65536U - T1_COUNTS_1MS) /* 65536-922=64614=0xFC66 */

static volatile unsigned long g_ms = 0;

void timer1_isr(void) interrupt 3
{
    TH1 = (unsigned char)(T1_RELOAD >> 8);  /* 0xFC */
    TL1 = (unsigned char)(T1_RELOAD & 0xFF);/* 0x66 */
    g_ms++;
}

static void systick_init(void)
{
    /* Timer1 Mode1 16-bit */
    TMOD = (TMOD & 0x0F) | 0x10;
```

```
    TH1 = (unsigned char)(T1_RELOAD >> 8);
    TL1 = (unsigned char)(T1_RELOAD & 0xFF);

    ET1 = 1;  /* enable Timer1 interrupt */
    EA  = 1;  /* global enable */
    TR1 = 1;  /* start Timer1 */
}

static unsigned long millis(void)
{
    unsigned long t;
    EA = 0;
    t = g_ms;
    EA = 1;
    return t;
}

/* ===================================================================== */
/* Buzzer output (Timer0) + Music state machine (NON-BLOCKING)         */
/* ===================================================================== */

/* 音调整体提升: freq *= 6/5  => half_period_us *= 5/6 */
#define PITCH_UP_NUM 5U
#define PITCH_UP_DEN 6U

static unsigned int pitch_half_period(unsigned int half_us)
{
    unsigned long t = (unsigned long)half_us * (unsigned long)PITCH_UP_NUM;
    t /= (unsigned long)PITCH_UP_DEN;
    if (t < 50UL) t = 50UL;
    return (unsigned int)t;
}

/* Timer0 reload for buzzer */
static volatile unsigned int t0_reload;
static volatile bit buzzer_running = 0;

void timer0_isr(void) interrupt 1
{
    unsigned int r = t0_reload;
    TH0 = (unsigned char)(r >> 8);
    TL0 = (unsigned char)(r & 0xFF);
    BUZZER = !BUZZER;
}

static void buzzer_set_half_period_us(unsigned int half_period_us)
```

```
    {
        /* counts = half_us * TIMER_TICK_HZ / 1e6 */
        unsigned long counts_ul = (unsigned long)half_period_us * (unsigned
long)TIMER_TICK_HZ;
        counts_ul /= 1000000UL;

        if (counts_ul < 1UL) counts_ul = 1UL;
        if (counts_ul > 60000UL) counts_ul = 60000UL;

        t0_reload = (unsigned int)(65536U - (unsigned int)counts_ul);
    }

    static void buzzer_start(void)
    {
        /* Timer0 Mode1 16-bit */
        TMOD = (TMOD & 0xF0) | 0x01;
        TH0 = (unsigned char)(t0_reload >> 8);
        TL0 = (unsigned char)(t0_reload & 0xFF);
        ET0 = 1;
        EA  = 1;
        TR0 = 1;
        buzzer_running = 1;
    }

    static void buzzer_stop(void)
    {
        TR0 = 0;
        ET0 = 0;
        BUZZER = 0;
        buzzer_running = 0;
    }

    /* Notes (half period us) */
    #define NOTE_C4 1911U
    #define NOTE_D4 1703U
    #define NOTE_E4 1517U
    #define NOTE_F4 1433U
    #define NOTE_G4 1275U
    #define NOTE_A4 1136U

    /* 时值：句尾长音稍微缩短一点点 */
    #define DUR_SHORT   400U
    #define DUR_LONG   1000U   /* 原 1200U -> 1000U */
    #define NOTE_GAP_MS   30U   /* 音与音之间间隔 */
    #define PHRASE_GAP_MS 90U   /* 句末额外停顿：120U -> 90U */

    static const unsigned int code melody[] = {
```

```
    /* Phrase 1: C C G G A A G */
    NOTE_C4, NOTE_C4, NOTE_G4, NOTE_G4, NOTE_A4, NOTE_A4, NOTE_G4,
    /* Phrase 2: F F E E D D C */
    NOTE_F4, NOTE_F4, NOTE_E4, NOTE_E4, NOTE_D4, NOTE_D4, NOTE_C4,
    /* Phrase 3: G G F F E E D */
    NOTE_G4, NOTE_G4, NOTE_F4, NOTE_F4, NOTE_E4, NOTE_E4, NOTE_D4,
    /* Phrase 4: C C G G A A G */
    NOTE_C4, NOTE_C4, NOTE_G4, NOTE_G4, NOTE_A4, NOTE_A4, NOTE_G4,
    /* Phrase 5: F F E E D D C */
    NOTE_F4, NOTE_F4, NOTE_E4, NOTE_E4, NOTE_D4, NOTE_D4, NOTE_C4
};

static const unsigned int code durations[] = {
    /* Phrase 1 */
    DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_LONG,
    /* Phrase 2 */
    DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_LONG,
    /* Phrase 3 */
    DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_LONG,
    /* Phrase 4 */
    DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_LONG,
    /* Phrase 5 */
    DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_SHORT, DUR_LONG
};

#define MELODY_LEN (sizeof(melody) / sizeof(melody[0]))

/* 音乐状态机 */
typedef enum {
    MUS_IDLE = 0,
    MUS_NOTE_ON,
    MUS_GAP
} mus_state_t;

static mus_state_t mus_state = MUS_IDLE;
static unsigned char mus_i = 0;
static unsigned long mus_deadline = 0; /* 到这个时间点（ms）执行下一步 */
static bit mus_screen_inited = 0;

static void music_reset(void)
{
    mus_state = MUS_IDLE;
    mus_i = 0;
    mus_deadline = 0;
    mus_screen_inited = 0;
    buzzer_stop();
}
```

```
/* 每次主循环调用：不阻塞 */
static void music_task(void)
{
    unsigned long now = millis();

    if (!mus_screen_inited) {
        LCD12864_Clear();
        LCD_ShowStr16x8(1, 0, (uint8_t *)"MUSC");
        LCD_ShowStr16x8(1, 32, (uint8_t *)"STAR");
        mus_screen_inited = 1;
    }

    switch (mus_state) {
    case MUS_IDLE:
        /* 开始第一音 */
        mus_i = 0;
        mus_state = MUS_NOTE_ON;
        /* fallthrough */

    case MUS_NOTE_ON:
        if (mus_i >= MELODY_LEN) {
            /* 播放完一遍，循环播放 */
            mus_i = 0;
        }
        /* 启动当前音 */
        {
            unsigned int hp = pitch_half_period(melody[mus_i]);
            buzzer_set_half_period_us(hp);
            if (!buzzer_running) buzzer_start();
            mus_deadline = now + (unsigned long)durations[mus_i];
            mus_state = MUS_GAP; /* 下一步进入 gap（先关蜂鸣器，再等间隔） */
        }
        break;

    case MUS_GAP:
        if (now >= mus_deadline) {
            /* 到点：关蜂鸣器，并设置“间隔结束时间” */
            buzzer_stop();

            /* 计算间隔：普通音 gap + 句末额外 gap */
            {
                unsigned int gap = NOTE_GAP_MS;
                if (((mus_i + 1) % 7) == 0) gap += PHRASE_GAP_MS;
                mus_deadline = now + (unsigned long)gap;
            }
```

```
            /* 进入“等待间隔结束”阶段：用 MUS_IDLE/MUS_NOTE_ON 不好区分，
             * 这里复用 MUS_GAP，但用 buzzer_running=0 作为“已关蜂鸣器”的标记 */
            mus_state = MUS_NOTE_ON;
            mus_i++; /* 下一音索引先+1，等间隔结束再触发下一音 */
            /* 为了让间隔也可计时，这里把 MUS_NOTE_ON 当作“等待间隔结束再起音” */
            /* 但 MUS_NOTE_ON 现在会立刻起音，所以我们用一个小技巧：deadline 表示 gap 结束点 */
            /* 于是增加一个额外判断：若 now < deadline，则不起音 */
        }
        break;

    default:
        mus_state = MUS_IDLE;
        break;
    }

    /* “等待间隔结束再起音”的逻辑：把 MUS_NOTE_ON 的动作延后 */
    if (mus_state == MUS_NOTE_ON) {
        /* 如果 deadline 还在未来，说明还在 gap 中，不启动下一音 */
        if (now < mus_deadline) {
            /* do nothing */
        } else {
            /* 现在可以启动下一音：借用 MUS_NOTE_ON 的启动逻辑 */
            if (mus_i >= MELODY_LEN) mus_i = 0;
            {
                unsigned int hp2 = pitch_half_period(melody[mus_i]);
                buzzer_set_half_period_us(hp2);
                if (!buzzer_running) buzzer_start();
                mus_deadline = now + (unsigned long)durations[mus_i];
                /* 用 MUS_GAP 代表“正在发声，等到 deadline 后关蜂鸣器并安排 gap” */
                mus_state = MUS_GAP;
            }
        }
    }
}

/* ==================================================================== */
/* main                                                                 */
/* ==================================================================== */

void main()
{
    uint8_t prev_mode = 0;

    LCD12864_Init();
    systick_init();         /* <<<<<< 1ms tick 启动 */
    music_reset();          /* 确保蜂鸣器初始关闭 */
```

```
    select_adc_channel(channel);
    CH_SEL = channel;

    while (1) {
        uint8_t i;
        uint8_t col, row, page;
        uint8_t Data, old_Data, old_page;
        uint8_t temp_Data2, temp_page;
        uint8_t t, d, k, tmp;

        uint8_t current_range_code;

        int maxv_mV, minv_mV, rmsv_mV;
        char numstr[6];
        uint8_t sample_value;
        int sample_mV;
        long sum_sq_mV;

        /* ===================== Key detection ===================== */
        if (KEY == 0) {
            key_count++;
            if (!long_press_handled && key_count >= KEY_THRESHOLD) {
                channel ^= 1;
                CH_SEL = !CH_SEL;
                select_adc_channel(channel);

                if (channel == 1) {
                    wave_index = 0;
                    current_range_code = ((uint8_t)RANGE1 << 1) | (uint8_t)RANGE0;
                    prev_range_code = current_range_code;
                    set_waveform_pins(wave_index);
                }
                LCD12864_Clear();
                long_press_handled = 1;
            }
        } else {
            if (key_count > 0 && key_count < KEY_THRESHOLD) {
                if (mode == 1) mode = 2;
                else if (mode == 2) mode = 3;
                else mode = 1;

                LCD12864_Clear();
            }
            key_count = 0;
            long_press_handled = 0;
        }
```

```
        /* 模式切换时的进入处理 */
        if (mode != prev_mode) {
            if (prev_mode == 3) {
                /* 离开音乐模式：立刻停止蜂鸣器并重置音乐状态 */
                music_reset();
            }
            if (mode == 3) {
                /* 进入音乐模式：重置，保证从头播放 */
                music_reset();
            }
            prev_mode = mode;
        }

        /* ===================== Mode handling ===================== */
        if (mode == 1) {
            /* Oscilloscope mode */
            old_Data = 0;
            old_page = 0;

            for (i = 0; i < COLLECT_NUM; i++) {
                sample_value = GatAdcValue();
                col = i;
                row = sample_value / 4;
                page = row / 8;
                Data = (uint8_t)(0x01 << (row - (page * 8)));

                refresh_one_list(col);

                if (i != 0) {
                    temp_Data2 = Data;
                    temp_page = page;

                    if (page == old_page) {
                        if (Data > old_Data) {
                            t = Data >> 1;
                            while (t != old_Data) {
                                t = t >> 1;
                                Data = (uint8_t)(Data | (Data >> 1));
                            }
                        } else if (Data < old_Data) {
                            t = Data << 1;
                            while (t != old_Data) {
                                t = t << 1;
                                Data = (uint8_t)(Data | (Data << 1));
                            }
                        }
                    } else if (page > old_page) {
```

```
            t = Data;
            for (k = 0; k < (row - (page * 8)); k++) t |= (t >> 1);
            write_one_list(page, col, t);
            page--;
            while (page != old_page) {
                write_one_list(page, col, 0xFF);
                page--;
            }
            if (page == old_page) {
                d = 0x80;
                tmp = 0x80;
                if (d > old_Data) {
                    tmp = tmp >> 1;
                    while (tmp != old_Data) {
                        tmp = tmp >> 1;
                        d = (uint8_t)(d | (d >> 1));
                    }
                }
                Data = d;
                page = old_page;
            }
        } else { /* page < old_page */
            t = Data;
            for (k = 0; k < (7 - (row - (page * 8))); k++) t |= (t << 1);
            write_one_list(page, col, t);
            page++;
            while (page != old_page) {
                write_one_list(page, col, 0xFF);
                page++;
            }
            if (page == old_page) {
                d = 0x01;
                tmp = 0x01;
                if (d < old_Data) {
                    tmp = tmp << 1;
                    while (tmp != old_Data) {
                        tmp = tmp << 1;
                        d = (uint8_t)(d | (d << 1));
                    }
                }
                Data = d;
                page = old_page;
            }
        }

        old_Data = temp_Data2;
        old_page = temp_page;
```

```
            } else {
                old_Data = Data;
                old_page = page;
            }

            write_one_list(page, col, Data);
        }

        if (channel == 0) LCD_ShowStr16x8(1, 96, (uint8_t *)"IN0");
        else              LCD_ShowStr16x8(1, 96, (uint8_t *)"IN1");

    } else if (mode == 2) {
        /* Voltmeter mode */
        maxv_mV = -32768;
        minv_mV =  32767;
        sum_sq_mV = 0;

        for (i = 0; i < COLLECT_NUM; i++) {
            sample_value = GatAdcValue();
            sample_mV = adc_to_mV(sample_value, channel);
            if (sample_mV > maxv_mV) maxv_mV = sample_mV;
            if (sample_mV < minv_mV) minv_mV = sample_mV;
            sum_sq_mV += (long)sample_mV * (long)sample_mV;
        }

        {
            unsigned long avg_sq = (unsigned long)sum_sq_mV / COLLECT_NUM;
            rmsv_mV = isqrt32ul(avg_sq);
        }

        LCD12864_Clear();

        if (channel == 1) {
            current_range_code = ((uint8_t)RANGE1 << 1) | (uint8_t)RANGE0;
            if (current_range_code != prev_range_code) {
                wave_index = (uint8_t)((wave_index + 1) & 0x03);
                set_waveform_pins(wave_index);
                prev_range_code = current_range_code;
            }
        }

        if (channel == 0) {
            uint8_t range_code = ((uint8_t)RANGE1 << 1) | (uint8_t)RANGE0;
            LCD_ShowStr16x8(1, 0, (uint8_t *)"IN0 ");
            if     (range_code == 1) LCD_ShowStr16x8(1, 32, (uint8_t *)"2X");
            else if (range_code == 2) LCD_ShowStr16x8(1, 32, (uint8_t *)"4X");
            else                      LCD_ShowStr16x8(1, 32, (uint8_t *)"1X");
```

```
            } else {
               LCD_ShowStr16x8(1, 0, (uint8_t *)"IN1 ");
               switch (wave_index) {
               case 0: LCD_ShowStr16x8(1, 32, (uint8_t *)"SQRW"); break;
               case 1: LCD_ShowStr16x8(1, 32, (uint8_t *)"SAWT"); break;
               case 2: LCD_ShowStr16x8(1, 32, (uint8_t *)"TRIA"); break;
               default: LCD_ShowStr16x8(1, 32, (uint8_t *)"SINE"); break;
               }
            }

            LCD_ShowStr16x8(2, 0, (uint8_t *)"MAX:");
            voltage_to_str(maxv_mV, numstr);
            LCD_ShowStr16x8(2, 32, (uint8_t *)numstr);

            LCD_ShowStr16x8(3, 0, (uint8_t *)"MIN:");
            voltage_to_str(minv_mV, numstr);
            LCD_ShowStr16x8(3, 32, (uint8_t *)numstr);

            LCD_ShowStr16x8(4, 0, (uint8_t *)"RMS:");
            voltage_to_str(rmsv_mV, numstr);
            LCD_ShowStr16x8(4, 32, (uint8_t *)numstr);

         } else {
            /* Music mode - NON-BLOCKING */
            music_task();
         }
      }
   }
```

----------LCD12864.c----------

```
   #include "LCD12864.h"
   //#include "font.h"









   static void LCD12864_delay(uint8_t t)
   {
      uint8_t j;
      for(j=0;j<t;j++);
   }


   /*
```

```
    状态检测，只有 12864 处于正常状态切不忙的情况
    才可以继续对 12864 进行操作。
*/
static void LCD12864_CheckStatus()
{
    bit status;
    DATA = 0x00;
    RS = 0;
    RW = 1;  /*  拉低为写数据，拉高为读数据  */
    do
    {
        EN = 1;
        LCD12864_delay(2);
        status = (bit)(DATA&0x80);
        EN = 0;

    }while(status);

}


/* command：指令数据  */
static void LCD12864_WriteCommand(uint8_t command)
{
    LCD12864_CheckStatus();

    RS = 0;        /* RS 都拉低时，D0-D7 上的数据作为指令被送入指令寄存器 */
    RW = 0;        /*  拉低为写数据，拉高为读数据  */
    EN = 0;
    DATA = command;        /* 写入指令 */
    LCD12864_delay(2);  /* 等待数据稳定 */
    EN = 1;         /* 写使能，D0-D7 上的数据允许被写入寄存器  */
    LCD12864_delay(2);  /* 等待数据稳定 */
    EN = 0;            /* 锁定数据，D0-D7 上的数据不再被允许被写入寄存器 */

}


/*  写数据 */
static void LCD12864_WriteData(uint8_t Data)
{
    LCD12864_CheckStatus();

    RS = 1;  /* 写数据，D0-D7 上的数据被送到显示 RAM */
    RW = 0;  /* 拉低为写数据，拉高为读数据  */
    EN = 0;
    DATA = Data;
```

```
    LCD12864_delay(2);  /* 等待数据稳定 */
    EN = 1;         /* 写使能，D0-D7 上的数据允许被写入寄存器  */
    LCD12864_delay(2);  /* 等待数据稳定 */
    EN = 0;            /* 锁定数据，D0-D7 上的数据不再被允许被写入寄存器 */

}



/*
    开显示
*/
void LCD12864_On()
{
    LCD12864_WriteCommand(LCD_ON);
}

/*
    关显示
*/
void LCD12864_Off()
{
    LCD12864_WriteCommand(LCD_OFF);
}









/* 设置列地址，0—127 列，地址依次递增 */
void LCD12864_SetColAdd(uint8_t offset)
{
    uint8_t temp = offset;

    if( 63<temp)
    {


        CS1 = 1;
        CS2 = 0;     /* 右半屏 */
        temp = temp - 64;
    }
    else
    {
        CS1 = 0;
        CS2 = 1;  /* 左半屏 */
```

```
    }

    LCD12864_WriteCommand(LCD_COL_ADD+temp);


}

/*
    设置行地址，0—63 行，地址依次递增

*/
void LCD12864_SetRowAdd(uint8_t offset)
{

    LCD12864_WriteCommand(LCD_ROW_ADD+offset);
}

/*
    设置页地址，0—7 页，地址依次递增

*/
void LCD12864_SetPageAdd(uint8_t offset)
{

    LCD12864_WriteCommand(LCD_PAGE_ADD+offset);
}


void LCD12864_Clear()
{
    uint8_t page_index = 0;
    uint8_t col_index = 0;

    CS1 = CS2 = 0;
    for(page_index = 0;page_index<8;page_index++)
    {
        LCD12864_SetPageAdd(page_index);
        for(col_index = 0;col_index<64; col_index++)
        {
            LCD12864_WriteCommand(LCD_COL_ADD+col_index);
            LCD12864_WriteData(0x00);
        }
    }

}

void LCD12864_Init()
```

```
{

    LCD12864_delay(2);
    CS1 = 1;
    CS2 = 1;   /* CS1、CS2 都为 0 是选择全屏 */
    LCD12864_delay(2);

    LCD12864_Off();
    LCD12864_SetPageAdd(0);
    LCD12864_SetRowAdd(0);
    LCD12864_SetColAdd(0);

    LCD12864_On();
    LCD12864_Clear();
}




/*
    ASCII 字符,16 个点高 8 个点宽
*/
//void LCD12864_ShowChar16x8(uint8_t page,uint8_t col,uint8_t ch)
//{
//    uint8_t j,i,list;
//    list  = col;
//    ch = ch-' ';
//
//    for(j = 0;j <8;j++)
//    {
//
//            for(i = 0;i<2;i++)
//            {
//                LCD12864_SetPageAdd(page+i);
//
//                LCD12864_SetColAdd(list+j);
//                LCD12864_WriteData(asc2_1608[ch][j*2+i]);
//
//            }
//    }
//}


/*
    英文字符显示  支持自动换行显示
    line:起始页，范围 1~4
    col :起始列，一个字体占用 8 列（宽 8 个点）。范围 0~127
```

```
*/
//void LCD12864_ShowStr16x8(uint8_t line,uint8_t col,uint8_t *ch)
//{
//    uint8_t page,list;
//    list = col;
//    switch(line)
//    {
//        case 1:page = 0;break;
//        case 2:page = 2;break;
//        case 3:page = 4;break;
//        case 4:page = 6;break;
//        default:    return;
//
//    }
//    while((*ch <= '~')&&(*ch>= ' '))   /*  判断是不是非法字符，只有 ASCII 码中的字符可以显示 */
//    {
//        if( list<128 )
//            LCD12864_ShowChar16x8(page,list,*ch);

//        list += 8;
//        if(list>127)     /* 列数超出了范围 */
//        {
//            page +=2;   /* 换行，一行字占两页 */
//            list = 0;    /* 从新的一行的第 0 列开始 */

//            if(page>6) return;   /* 底部超出范围，无法继续显示 */
//        }
//        ch++;
//    }
//}

/*
    汉字显示 16*16 格式  支持自动换行显示
    汉字取模软件用的是 PCtoLCD2002 完美版
    取模方式：阴码、逆向、c51 格式、逐列式、十六进制数

    line:起始页，范围 1~4
    col :起始列，一个字体占用 8 列（宽 8 个点）。范围 0~127
    Hz :字模数组的首地址
    Hz_size:  Hz 数组所占用的字节数，等于 sizeof(Hz)
*/
//void LCD12864_ShowHz(uint8_t line,uint8_t col,uint8_t *Hz,uint16_t Hz_size)
//{
//    uint8_t page,i;
//    uint16_t num,j;
//    uint8_t col_temp;
//    col_temp = col;
```

```
//    num = Hz_size/2;
//
//    switch(line)
//    {
//       case 1:page = 0;break;
//       case 2:page = 2;break;
//       case 3:page = 4;break;
//       case 4:page = 6;break;
//       default:   return;
//    }
//
//
//    for(j = 0;j <num;j++)
//    {
//       for(i = 0;i<2;i++)
//       {
//          LCD12864_SetPageAdd(page+i);

//          LCD12864_SetColAdd(col_temp);
//          LCD12864_WriteData(Hz[j*2+i]);
//       }
//       col_temp++;
//       if(col_temp>127)
//       {
//          col_temp-=128;
//          page+=2;
//          if(page>6)
//             return;
//       }
//    }
//
//}


void refresh_one_list(uint8_t col)
{
   uint8_t i;

   for(i = 0;i<8;i++)
   {

      LCD12864_SetPageAdd(i);
      LCD12864_SetColAdd(col);
      LCD12864_WriteData(0x00);
   }
}
```

```
   void write_one_list(uint8_t page,uint8_t col,uint8_t Data)
   {

      LCD12864_SetPageAdd(page);
      LCD12864_SetColAdd(col);
      LCD12864_WriteData(Data);

   }
-----------------------------------LCD12864.h----------------------------------------
#ifndef __LCD12864_H
#define __LCD12864_H

#include "reg51.h"

typedef unsigned char uint8_t;
typedef unsigned short uint16_t;
typedef unsigned int uint32_t;

sbit RS  = P2^2;
sbit RW  = P2^1;
sbit EN  = P2^0;
sbit CS1 = P2^4;
sbit CS2 = P2^3;


#define DATA  P1


#define  LCD_COL_ADD  0x40    /* 列的起始地址（第一列的地址）(0—63 列，地址依次递增）*/
#define  LCD_ROW_ADD  0xC0    /* 行的起始地址（第一行的地址）(0—63 行，地址依次递增) */
#define  LCD_PAGE_ADD 0xB8    /* 页的起始地址（第一页的地址）(0—7 页，地址依次递增） */


#define  LCD_ON      0x3F   /* 开显示，不影响显示 RAM(DD RAM)中的内容。*/
#define  LCD_OFF     0x3E   /* 关显示，不影响显示 RAM(DD RAM)中的内容。*/



void LCD12864_on(void);
void LCD12864_Off(void);

void LCD12864_SetColAdd(uint8_t offset);
void LCD12864_SetRowAdd(uint8_t offset);
void LCD12864_SetPageAdd(uint8_t offset);
void LCD12864_Init(void);
//void LCD12864_ShowChar16x8(uint8_t page,uint8_t col,uint8_t ch);
```

```
//void LCD12864_ShowStr16x8(uint8_t line,uint8_t col,uint8_t *ch);
//void LCD12864_ShowHz(uint8_t line,uint8_t col,uint8_t *Hz,uint16_t Hz_size);
void write_one_list(uint8_t page,uint8_t col,uint8_t Data);
void LCD12864_Clear();

void refresh_one_list(uint8_t col);



#endif
--------------------------------------font.c-----------------------------------------
#include "font.h"


//16*16 ASCII 字符集点阵
unsigned char code asc2_1608[95][16]={
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*" ",0*/
{0x00,0x00,0x00,0x00,0x00,0x00,0xF8,0x33,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*"!",1*/
{0x00,0x00,0x10,0x00,0x0C,0x00,0x02,0x00,0x10,0x00,0x0C,0x00,0x02,0x00,0x00,0x00},
/*"""",2*/
{0x00,0x00,0x40,0x04,0xC0,0x3F,0x78,0x04,0x40,0x04,0xC0,0x3F,0x78,0x04,0x00,0x00},
/*"#",3*/
{0x00,0x00,0x70,0x18,0x88,0x20,0x88,0x20,0xFC,0xFF,0x08,0x21,0x30,0x1E,0x00,0x00},
/*"$",4*/
{0xF0,0x00,0x08,0x31,0xF0,0x0C,0x80,0x03,0x60,0x1E,0x18,0x21,0x00,0x1E,0x00,0x00},
/*"%",5*/
{0x00,0x1E,0xF0,0x21,0x08,0x23,0x88,0x2C,0x70,0x19,0x00,0x27,0x00,0x21,0x00,0x10},
/*"&",6*/
{0x00,0x00,0x12,0x00,0x0E,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*"'",7*/
{0x00,0x00,0x00,0x00,0x00,0x00,0xE0,0x07,0x18,0x18,0x04,0x20,0x02,0x40,0x00,0x00},
/*"(",8*/
{0x00,0x00,0x02,0x40,0x04,0x20,0x18,0x18,0xE0,0x07,0x00,0x00,0x00,0x00,0x00,0x00},
/*")",9*/
{0x40,0x02,0x40,0x02,0x80,0x01,0xF0,0x0F,0x80,0x01,0x40,0x02,0x40,0x02,0x00,0x00},
/*"*",10*/
{0x00,0x00,0x00,0x01,0x00,0x01,0x00,0x01,0xE0,0x0F,0x00,0x01,0x00,0x01,0x00,0x01},
/*"+",11*/
{0x00,0x00,0x00,0x90,0x00,0x70,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*",",12*/
{0x00,0x00,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x01,0x00,0x00},
/*"-",13*/
{0x00,0x00,0x00,0x30,0x00,0x30,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*".",14*/
{0x00,0x00,0x00,0x60,0x00,0x18,0x00,0x07,0xC0,0x00,0x38,0x00,0x04,0x00,0x00,0x00}
```

```
,/*"/",15*/
{0x00,0x00,0xE0,0x0F,0x10,0x10,0x08,0x20,0x08,0x20,0x10,0x10,0xE0,0x0F,0x00,0x00},
/*"0",16*/
{0x00,0x00,0x00,0x00,0x10,0x20,0x10,0x20,0xF8,0x3F,0x00,0x20,0x00,0x20,0x00,0x00},
/*"1",17*/
{0x00,0x00,0x70,0x30,0x08,0x28,0x08,0x24,0x08,0x22,0x08,0x21,0xF0,0x30,0x00,0x00},
/*"2",18*/
{0x00,0x00,0x30,0x18,0x08,0x20,0x08,0x21,0x08,0x21,0x88,0x22,0x70,0x1C,0x00,0x00},
/*"3",19*/
{0x00,0x00,0x00,0x06,0x80,0x05,0x40,0x24,0x30,0x24,0xF8,0x3F,0x00,0x24,0x00,0x24},
/*"4",20*/
{0x00,0x00,0xF8,0x19,0x88,0x20,0x88,0x20,0x88,0x20,0x08,0x11,0x08,0x0E,0x00,0x00},
/*"5",21*/
{0x00,0x00,0xE0,0x0F,0x10,0x11,0x88,0x20,0x88,0x20,0x90,0x20,0x00,0x1F,0x00,0x00},
/*"6",22*/
{0x00,0x00,0x18,0x00,0x08,0x00,0x08,0x3E,0x88,0x01,0x68,0x00,0x18,0x00,0x00,0x00},
/*"7",23*/
{0x00,0x00,0x70,0x1C,0x88,0x22,0x08,0x21,0x08,0x21,0x88,0x22,0x70,0x1C,0x00,0x00},
/*"8",24*/
{0x00,0x00,0xF0,0x01,0x08,0x12,0x08,0x22,0x08,0x22,0x10,0x11,0xE0,0x0F,0x00,0x00},
/*"9",25*/
{0x00,0x00,0x00,0x00,0x00,0x00,0xC0,0x30,0xC0,0x30,0x00,0x00,0x00,0x00,0x00,0x00},
/*":",26*/
{0x00,0x00,0x00,0x00,0x00,0x00,0x80,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*";",27*/
{0x00,0x00,0x00,0x01,0x80,0x02,0x40,0x04,0x20,0x08,0x10,0x10,0x08,0x20,0x00,0x00},
/*"<",28*/
{0x00,0x00,0x40,0x02,0x40,0x02,0x40,0x02,0x40,0x02,0x40,0x02,0x40,0x02,0x00,0x00},
/*"=",29*/
{0x00,0x00,0x08,0x20,0x10,0x10,0x20,0x08,0x40,0x04,0x80,0x02,0x00,0x01,0x00,0x00},
/*">",30*/
{0x00,0x00,0x70,0x00,0x48,0x00,0x08,0x30,0x08,0x37,0x88,0x00,0x70,0x00,0x00,0x00},
/*"?",31*/
{0xC0,0x07,0x30,0x18,0xC8,0x27,0x28,0x28,0xE8,0x2F,0x10,0x28,0xE0,0x17,0x00,0x00},
/*"@",32*/
{0x00,0x20,0x00,0x3C,0xC0,0x23,0x38,0x02,0xE0,0x02,0x00,0x27,0x00,0x38,0x00,0x20},
/*"A",33*/
{0x08,0x20,0xF8,0x3F,0x88,0x20,0x88,0x20,0x88,0x20,0x70,0x11,0x00,0x0E,0x00,0x00},
/*"B",34*/
{0xC0,0x07,0x30,0x18,0x08,0x20,0x08,0x20,0x08,0x20,0x08,0x10,0x38,0x08,0x00,0x00},
/*"C",35*/
{0x08,0x20,0xF8,0x3F,0x08,0x20,0x08,0x20,0x08,0x20,0x10,0x10,0xE0,0x0F,0x00,0x00},
/*"D",36*/
{0x08,0x20,0xF8,0x3F,0x88,0x20,0x88,0x20,0xE8,0x23,0x08,0x20,0x10,0x18,0x00,0x00},
/*"E",37*/
{0x08,0x20,0xF8,0x3F,0x88,0x20,0x88,0x00,0xE8,0x03,0x08,0x00,0x10,0x00,0x00,0x00},
/*"F",38*/
```

```
{0xC0,0x07,0x30,0x18,0x08,0x20,0x08,0x20,0x08,0x22,0x38,0x1E,0x00,0x02,0x00,0x00},
/*"G",39*/
{0x08,0x20,0xF8,0x3F,0x08,0x21,0x00,0x01,0x00,0x01,0x08,0x21,0xF8,0x3F,0x08,0x20},
/*"H",40*/
{0x00,0x00,0x08,0x20,0x08,0x20,0xF8,0x3F,0x08,0x20,0x08,0x20,0x00,0x00,0x00,0x00},
/*"I",41*/
{0x00,0xC0,0x00,0x80,0x08,0x80,0x08,0x80,0xF8,0x7F,0x08,0x00,0x08,0x00,0x00,0x00},
/*"J",42*/
{0x08,0x20,0xF8,0x3F,0x88,0x20,0xC0,0x01,0x28,0x26,0x18,0x38,0x08,0x20,0x00,0x00},
/*"K",43*/
{0x08,0x20,0xF8,0x3F,0x08,0x20,0x00,0x20,0x00,0x20,0x00,0x20,0x00,0x30,0x00,0x00},
/*"L",44*/
{0x08,0x20,0xF8,0x3F,0xF8,0x01,0x00,0x3E,0xF8,0x01,0xF8,0x3F,0x08,0x20,0x00,0x00},
/*"M",45*/
{0x08,0x20,0xF8,0x3F,0x30,0x20,0xC0,0x00,0x00,0x07,0x08,0x18,0xF8,0x3F,0x08,0x00},
/*"N",46*/
{0xE0,0x0F,0x10,0x10,0x08,0x20,0x08,0x20,0x08,0x20,0x10,0x10,0xE0,0x0F,0x00,0x00},
/*"O",47*/
{0x08,0x20,0xF8,0x3F,0x08,0x21,0x08,0x01,0x08,0x01,0x08,0x01,0xF0,0x00,0x00,0x00},
/*"P",48*/
{0xE0,0x0F,0x10,0x10,0x08,0x28,0x08,0x28,0x08,0x30,0x10,0x50,0xE0,0x4F,0x00,0x00},
/*"Q",49*/
{0x08,0x20,0xF8,0x3F,0x88,0x20,0x88,0x00,0x88,0x03,0x88,0x0C,0x70,0x30,0x00,0x20},
/*"R",50*/
{0x00,0x00,0x70,0x38,0x88,0x20,0x08,0x21,0x08,0x21,0x08,0x22,0x38,0x1C,0x00,0x00},
/*"S",51*/
{0x18,0x00,0x08,0x00,0x08,0x20,0xF8,0x3F,0x08,0x20,0x08,0x00,0x18,0x00,0x00,0x00},
/*"T",52*/
{0x08,0x00,0xF8,0x1F,0x08,0x20,0x00,0x20,0x00,0x20,0x08,0x20,0xF8,0x1F,0x08,0x00},
/*"U",53*/
{0x08,0x00,0x78,0x00,0x88,0x07,0x00,0x38,0x00,0x0E,0xC8,0x01,0x38,0x00,0x08,0x00},
/*"V",54*/
{0x08,0x00,0xF8,0x03,0x00,0x3E,0xF8,0x01,0x00,0x3E,0xF8,0x03,0x08,0x00,0x00,0x00},
/*"W",55*/
{0x08,0x20,0x18,0x30,0x68,0x2C,0x80,0x03,0x80,0x03,0x68,0x2C,0x18,0x30,0x08,0x20},
/*"X",56*/
{0x08,0x00,0x38,0x00,0xC8,0x20,0x00,0x3F,0xC8,0x20,0x38,0x00,0x08,0x00,0x00,0x00},
/*"Y",57*/
{0x10,0x20,0x08,0x38,0x08,0x26,0x08,0x21,0xC8,0x20,0x38,0x20,0x08,0x18,0x00,0x00},
/*"Z",58*/
{0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0x7F,0x02,0x40,0x02,0x40,0x02,0x40,0x00,0x00},
/*"[",59*/
{0x00,0x00,0x04,0x00,0x38,0x00,0xC0,0x01,0x00,0x06,0x00,0x38,0x00,0xC0,0x00,0x00},
/*"\",60*/
{0x00,0x00,0x02,0x40,0x02,0x40,0x02,0x40,0xFE,0x7F,0x00,0x00,0x00,0x00,0x00,0x00},
/*"]",61*/
{0x00,0x00,0x00,0x00,0x04,0x00,0x02,0x00,0x02,0x00,0x04,0x00,0x00,0x00,0x00,0x00},
```

```
/*"^",62*/
{0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80,0x00,0x80},
/*"_",63*/
{0x00,0x00,0x02,0x00,0x02,0x00,0x04,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*"`",64*/
{0x00,0x00,0x00,0x19,0x80,0x24,0x80,0x24,0x80,0x12,0x00,0x3F,0x00,0x20,0x00,0x00},
/*"a",65*/
{0x10,0x00,0xF0,0x3F,0x00,0x11,0x80,0x20,0x80,0x20,0x00,0x11,0x00,0x0E,0x00,0x00},
/*"b",66*/
{0x00,0x00,0x00,0x0E,0x00,0x11,0x80,0x20,0x80,0x20,0x80,0x20,0x00,0x11,0x00,0x00},
/*"c",67*/
{0x00,0x00,0x00,0x1F,0x80,0x20,0x80,0x20,0x80,0x20,0x90,0x10,0xF0,0x3F,0x00,0x20},
/*"d",68*/
{0x00,0x00,0x00,0x1F,0x80,0x24,0x80,0x24,0x80,0x24,0x80,0x24,0x00,0x17,0x00,0x00},
/*"e",69*/
{0x00,0x00,0x80,0x20,0x80,0x20,0xE0,0x3F,0x90,0x20,0x90,0x20,0x20,0x00,0x00,0x00},
/*"f",70*/
{0x00,0x00,0x00,0x6B,0x80,0x94,0x80,0x94,0x80,0x94,0x80,0x93,0x80,0x60,0x00,0x00},
/*"g",71*/
{0x10,0x20,0xF0,0x3F,0x00,0x21,0x80,0x00,0x80,0x00,0x80,0x20,0x00,0x3F,0x00,0x20},
/*"h",72*/
{0x00,0x00,0x80,0x20,0x98,0x20,0x98,0x3F,0x00,0x20,0x00,0x20,0x00,0x00,0x00,0x00},
/*"i",73*/
{0x00,0x00,0x00,0xC0,0x00,0x80,0x80,0x80,0x98,0x80,0x98,0x7F,0x00,0x00,0x00,0x00},
/*"j",74*/
{0x10,0x20,0xF0,0x3F,0x00,0x24,0x00,0x06,0x80,0x29,0x80,0x30,0x80,0x20,0x00,0x00},
/*"k",75*/
{0x00,0x00,0x10,0x20,0x10,0x20,0xF8,0x3F,0x00,0x20,0x00,0x20,0x00,0x00,0x00,0x00},
/*"l",76*/
{0x80,0x20,0x80,0x3F,0x80,0x20,0x80,0x00,0x80,0x3F,0x80,0x20,0x80,0x00,0x00,0x3F},
/*"m",77*/
{0x80,0x20,0x80,0x3F,0x00,0x21,0x80,0x00,0x80,0x00,0x80,0x20,0x00,0x3F,0x00,0x20},
/*"n",78*/
{0x00,0x00,0x00,0x1F,0x80,0x20,0x80,0x20,0x80,0x20,0x80,0x20,0x00,0x1F,0x00,0x00},
/*"o",79*/
{0x80,0x80,0x80,0xFF,0x00,0x91,0x80,0x20,0x80,0x20,0x00,0x11,0x00,0x0E,0x00,0x00},
/*"p",80*/
{0x00,0x00,0x00,0x0E,0x00,0x11,0x80,0x20,0x80,0x20,0x00,0x91,0x80,0xFF,0x00,0x80},
/*"q",81*/
{0x80,0x20,0x80,0x20,0x80,0x3F,0x00,0x21,0x80,0x20,0x80,0x00,0x80,0x01,0x00,0x00},
/*"r",82*/
{0x00,0x00,0x00,0x33,0x80,0x24,0x80,0x24,0x80,0x24,0x80,0x24,0x80,0x19,0x00,0x00},
/*"s",83*/
{0x00,0x00,0x80,0x00,0x80,0x00,0xE0,0x1F,0x80,0x20,0x80,0x20,0x00,0x10,0x00,0x00},
/*"t",84*/
{0x80,0x00,0x80,0x1F,0x00,0x20,0x00,0x20,0x00,0x20,0x80,0x10,0x80,0x3F,0x00,0x20},
/*"u",85*/
```

```
{0x80,0x00,0x80,0x03,0x80,0x0C,0x00,0x30,0x80,0x0C,0x80,0x03,0x80,0x00,0x00,0x00},
/*"v",86*/
{0x80,0x01,0x80,0x0E,0x00,0x30,0x80,0x0C,0x80,0x07,0x00,0x38,0x80,0x06,0x80,0x01},
/*"w",87*/
{0x00,0x00,0x80,0x20,0x80,0x31,0x80,0x0E,0x00,0x2E,0x80,0x31,0x80,0x20,0x00,0x00},
/*"x",88*/
{0x80,0x00,0x80,0x81,0x80,0x86,0x00,0x78,0x00,0x18,0x80,0x06,0x80,0x01,0x80,0x00},
/*"y",89*/
{0x00,0x00,0x80,0x21,0x80,0x30,0x80,0x2C,0x80,0x22,0x80,0x21,0x80,0x30,0x00,0x00},
/*"z",90*/
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0xFC,0x3E,0x02,0x40,0x02,0x40},
/*"{",91*/
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0xFF,0x00,0x00,0x00,0x00,0x00,0x00},
/*"|",92*/
{0x02,0x40,0x02,0x40,0xFC,0x3E,0x00,0x01,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
/*"}",93*/
{0x00,0x00,0x02,0x00,0x01,0x00,0x02,0x00,0x02,0x00,0x04,0x00,0x02,0x00,0x00,0x00},
/*"~",94*/
};
```